Università di Genova
DIBRIS DIPARTIMENTO
DI INFORMATICA, BIOINGEGNERIA,
ROBOTICA E INGEGNERIA DEI SISTEMI

Template for Master's Theses at DIBRIS

Matteo Dell'Amico

Master's Thesis

Università di Genova, DIBRIS
Via Dodecaneso, 35 16146 Genova, Italy
https://www.dibris.unige.it/



Computer Science MSc
Software Security and Engineering Curriculum

Template for Master's Theses at DIBRIS

Matteo Dell'Amico

Advisor: Advisor Name

Examiner: Examiner Name

October, 2026

Abstract

This thesis proposes the design and implementation of a domain-specific language (DSL) for specifying hook-based dynamic binary instrumentation in Windows environments. The work will evaluate freely available Windows hooking libraries, select one or more as compilation targets, and assess the resulting framework for performance measurement, security policy enforcement, and runtime monitoring. Particular attention will be given to thread safety and to reliable attachment and detachment from running processes in production settings. Candidates must be fluent in x86 assembly and have passed the BASC exam.

Table of Contents

Chapter 1 Introduction **7**

Chapter 2 Background **8**

2.1 The Application Binary Interface (ABI) 8

2.2 Dynamic-Link Libraries 10

2.2.1 Static vs. Dynamic Linking 10

2.2.2 Address Space Isolation and Virtual Memory Mapping 11

2.2.3 DLL Injection and Instrumentation Lifecycle 11

2.3 Paradigms of DBI 12

2.3.1 Execution Context and CPU State Preservation 12

2.4 Inline Hooking 13

2.4.1 Signature-Enforced Hooking: Microsoft Detours and MinHook . . . 14

2.4.2 Context-Aware Hooking: Frida-Gum 15

2.4.3 Hook Execution Flow 18

2.4.4 Instruction Boundaries and the Role of Disassemblers 21

2.4.5 Transactional Patching 22

2.4.6 Thread-Safe Ejection 24

2.4.7 Summary of Inline Hooking Frameworks 26

2.5 Code-Cache-Based DBI Architectures 27

2.5.1 Core Architectural Components 27

2.5.2 The Execution Lifecycle: The JIT Loop . . . 28
2.5.3 The Client Interface: Instrumentation vs. Analysis Functions . . . 29
2.5.4 High-Level Abstraction API: Intel PIN . . . 30
2.5.5 low-level API: DynamoRIO . . . 31
2.5.6 Summary of Code-Cache-Based DBI Frameworks . . . 32
2.6 Comparison of Approaches . . . 33
2.7 Experimental Validation . . . 33
2.7.1 Scenario 1: Single-Threaded Hooking and Process Injection/Ejection Mechanics . . . 34
2.7.2 Scenario 2: Multi-Threaded Hooking and Concurrency Management 35
2.7.3 Scenario 3: Instruction-Level Assembly Tracking via Code-Cache-Based DBI Engines . . . 36

Chapter 3 Methodology **37**

3.1 Design Principles and Architecture . . . 37
3.1.1 Design Principles . . . 38
3.1.2 The Seven-Stage Pipeline . . . 39
3.1.3 The HookDesigner Architecture . . . 40
3.1.4 Selection of Compilation Targets . . . 41
3.2 Declarative Policy Specification and Logic File Integration . . . 42
3.2.1 The Core YAML Policy DSL . . . 42
3.2.2 Logic File Integration . . . 45
3.3 C++ Code Generation and DLL Compilation . . . 49
3.3.1 Engine Registry . . . 49
3.3.2 Jinja2 template architecture . . . 51
3.3.3 Target Address Resolution . . . 54
3.3.4 Hook Function Generation . . . 54
3.3.5 Hook Lifecycle Function Generation . . . 55

3.3.6 DLL Compilation . . . 58
3.4 HookDesigner Telemetry . . . 59
3.4.1 Telemetry Policy Specification . . . 60
3.4.2 Static Telemetry Expression Translation . . . 66
3.4.3 Static Semantic Validation . . . 71
3.4.4 Asynchronous Logging . . . 72
3.4.5 Performance Metrics Engine . . . 74
3.5 DLL Runtime Safety . . . 78
3.5.1 Asynchronous Exception Handling . . . 79
3.5.2 Transpiler Type Analysis and Memory Access Helpers . . . 83
3.5.3 Recursion Guard . . . 86
3.5.4 The Complete Hook Function Structure . . . 89
3.6 DLL Lifecycle and Deployment . . . 94
3.6.1 DLL Injection . . . 95
3.6.2 Injector Launch Mode . . . 99
3.6.3 DLL Ejection . . . 101

Chapter 4 Results **109**
4.1 Discussion . . . 110

Chapter 5 Conclusion **111**

Chapter 1

Introduction

At its core, dynamic binary instrumentation is the technique of inserting additional code into an existing, compiled executable to observe or modify its runtime behavior without requiring access to the original source code. This capability is critical for enforcing security policies and monitoring performance.

Advisor: Devo ancora sistemare questo capitolo

Chapter 2

Background

This chapter begins by introducing the foundational concepts of dynamic binary instrumentation (DBI) within a Windows environment necessary to understand the proposed work. Subsequently, it analyzes state-of-the-art frameworks utilized for DBI. This overview is essential for contextualizing the compilation targets selected for our Domain-Specific Language (DSL), which are discussed in detail in Methodology (Chapter 3).

As discussed in Introduction (Chapter 1), DBI allows us to change the runtime behavior of a binary, injecting a compiled payload into an existing executable.

2.1 The Application Binary Interface (ABI)

Ensuring interoperability between distinct compiled modules—regardless of the specific compilers or programming languages used—requires a *standardized low-level interface*. To allow instrumentation payloads to safely interact with a target process without causing unexpected runtime crashes, it must strictly adhere to the Application Binary Interface (ABI). The ABI defines the low-level rules for how functions interact at the assembly level, including how parameters are passed to a function, how return values are retrieved, which entity is responsible for cleaning up the stack (the caller or the callee), and the *register preservation contract* that dictates which registers must remain preserved across function boundaries. These rules are formalized as *Calling Conventions*. The complexity of ABI compliance and the risk of runtime crashes depend heavily on the target architecture.

Windows x86 (32-bit) Architecture

In 32-bit environments, multiple calling conventions coexist [Mic26b], requiring developer-level alignment between the instrumentation target function and the injected payload implementation. The most common conventions include:

- `__cdecl`: Arguments are pushed onto the stack from right to left, and the *caller* cleans the stack. This is the default convention for C/C++ binaries.
- `__stdcall`: The standard convention for the Windows application programming interface (API). Arguments are pushed right to left, but the *callee* cleans the stack.
- `__fastcall`: The first two arguments are passed in the `ECX` and `EDX` registers for speed, while the rest are placed on the stack.
- `__vectorcall`: An extended convention designed for SIMD processing, passing integer arguments in `ECX`/`EDX` and up to six vector arguments in `XMM0`–`XMM5`

Across these 32-bit conventions, registers are typically divided into *scratch registers* (`EAX`, `ECX`, and `EDX`), which the caller assumes can be freely overwritten by the called function, and *callee-saved registers* (`EBX`, `ESI`, `EDI`, and `EBP`), which the called function is contractually required to preserve and restore before returning. The `__vectorcall` convention changes this splitting by expanding the scratch registers set to include vector registers. Understanding and matching these different conventions is critical for DBI on x86.

Windows x64 (64-bit) Architecture

To eliminate the complexity of managing multiple incompatible conventions, Windows x64 introduces a single, unified calling convention (a standard x64 ABI) [Mic26e]. In this environment, legacy keywords like `__cdecl`, `__stdcall`, and `__fastcall` are accepted by the compiler for backward compatibility but are syntactically ignored. The x64 calling convention utilizes a register-based parameter passing scheme alongside a strict register preservation contract:

- The first four integer or pointer arguments are passed from left to right in registers (`RCX`, `RDX`, `R8`, and `R9`).
- The first four floating-point arguments are passed in the `XMM0`–`XMM3` registers.
- Any arguments beyond the first four are pushed onto the stack.

- The caller is always responsible for stack cleanup and must allocate 32 bytes of shadow store (home space) on the stack immediately before issuing a call.
- **Register Preservation Contract (Volatile vs. Non-Volatile):** Registers are strictly partitioned by caller/callee preservation responsibilities:
  - *Volatile (Caller-Saved):* Registers `RAX`, `RCX`, `RDX`, `R8`–`R11`, and `XMM0`–`XMM5` are designated as scratch registers. The caller must consider that these registers could be changed across a call; if the caller requires their values after the function returns, it must save them to its own stack frame prior to invoking the call.
  - *Non-Volatile (Callee-Saved):* Registers `RBX`, `RBP`, `RDI`, `RSI`, `R12`–`R15`, and `XMM6`–`XMM15` must remain preserved across calls. If a called routine needs to utilize any non-volatile register, it is contractually obligated to save the original value in the function prologue (typically via stack pushes) and restore it in the epilogue prior to returning.

From the perspective of DBI, the single-ABI model of x64 provides an inherent safety net: because the compiler forces both the target process and the injected payload to use the exact same assembly-level rules, calling convention mismatches are impossible. However, when writing portable instrumentation code intended to be compiled for both architectures, developers must still explicitly specify calling conventions (e.g., using macros like `WINAPI`) to prevent stack and registers corruption when the codebase is built for x86 targets.

2.2 Dynamic-Link Libraries

To perform DBI in modern Windows environments, external instrumentation code must execute directly within the target application's execution context. In Windows, this is commonly achieved through Dynamic-Link Libraries (dynamic-link librarys (DLLs)) and process injection.

2.2.1 Static vs. Dynamic Linking

Under the *static linking model*, all external functions and dependencies required by an application are permanently integrated into the final executable (EXE) file at link time. While this ensures that the application is fully self-contained, it introduces severe inefficiencies: every program on disk contains redundant copies of library code, and multiple running programs allocate separate physical RAM pages for the same helper routines.

Dynamic linking instead defers the binding of external library functions until runtime [Lev00, Ch.10]. A DLL is an independent binary executable module that contains

functions, resources, and data that can be called dynamically by other modules or applications. Rather than duplicating library code inside every host application on disk, the operating system maintains a single standalone DLL file. When multiple processes load the library, the operating system loader maps the same physical code pages in RAM into each process's virtual address space, drastically reducing both disk footprint and physical memory consumption.

2.2.2 Address Space Isolation and Virtual Memory Mapping

In modern operating systems, every running application is allocated an isolated, private virtual address space. This architectural boundary prevents one application from directly reading, writing, or corrupting the code or memory space of another.

To enable shared libraries to execute without violating this isolation model, the operating system loader utilizes *virtual memory mapping*. When a process loads a DLL, the loader maps the linear address range of the compiled DLL binary directly into the virtual address space of that process. Once mapped, functions within the DLL execute natively within the host application's memory context, allowing them to directly read, write, and manipulate the target's memory and execution state.

2.2.3 DLL Injection and Instrumentation Lifecycle

Because virtual memory mapping allows a DLL to execute inside a process's private address space, DLLs represent the primary vehicle for Windows DBI. However, because target applications are unaware of our instrumentation, the framework must force the target to load the payload from the outside. This is achieved through *DLL Injection* [The26a]. The standard injection technique leverages the Windows API functions `VirtualAllocEx`, `WriteProcessMemory`, and `CreateRemoteThread` targeting `LoadLibrary`. The injector process allocates memory in the remote target, writes the absolute path of the instrumentation DLL, and spawns a remote thread pointing to `LoadLibrary`, which executes within the target process and triggers the OS loader to map the DLL. Once mapped into the target process, DLLs provide an automatic lifecycle management mechanism via the entry point function, `DllMain` (code 2.1):

1. *Automatic Initialization (`DLL_PROCESS_ATTACH`):* Invoked automatically by the Windows loader immediately after the DLL is mapped into the process, allowing the framework to resolve target APIs and apply hooks.

2. *Automatic Cleanup (`DLL_PROCESS_DETACH`):* Invoked automatically when the DLL is unmapped (e.g., during module ejection via `FreeLibrary`), allowing the framework

to safely revert patches and deinitialize resources.

Code 2.1: DllMain code for attaching/detaching instrumentation

```
BOOL APIENTRY DllMain (HMODULE hModule,
DWORD   ul_reason_for_call,
LPVOID  lpReserved)
{
  switch (ul_reason_for_call)
  {
    case DLL_PROCESS_ATTACH:
    // Instrumentation framework-specific logic for injecting instrumentation
    break;
    case DLL_PROCESS_DETACH:
    // Instrumentation framework-specific logic for disabling instrumentation
    break;
  }

  return TRUE;
}
```

2.3 Paradigms of DBI

Modern DBI engines generally fall into two broad architectural paradigms:

- *Inline Hooking:* this paradigm involves modifying the original execution flow via localized jumps. It patches specific functions in memory, redirecting execution to a custom payload before returning to the original flow.
- *Code-Cache-Based DBI:* rather than modifying the original binary in place, DBI engines copy, recompile, and execute instructions inside a localized sandbox known as a Code Cache.

2.3.1 Execution Context and CPU State Preservation

Building upon the constraints of the ABI Section 2.1, it is necessary to formally define what constitutes the execution context of a running process and why preserving it is paramount to process stability during any paradigm of DBI.

At any given nanosecond, the exact state of an executing thread is represented by the data held within the CPU's internal registers. When an instrumentation framework intercepts

the execution flow, it redirects the instruction pointer to the user's custom analysis payload. Because this payload is a compiled piece of software itself, it inevitably utilizes CPU registers to perform its own calculations, memory allocations, or logging.

Under the calling convention rules of the ABI, the caller explicitly relies on the callee (the injected payload) to uphold the register preservation contract. While the caller expects its volatile registers to be modified across the call, it strictly depends on the callee to preserve its *non-volatile registers*. If the injected payload (that act as the callee) fails to strictly respect the target function's calling convention and signature, this contractual guarantee collapses. The compiler will generate prologue and epilogue instructions based on invalid assumptions, failing to save and restore the caller's non-volatile registers or mismanaging the stack layout. This introduces a critical point of failure known as *register corruption*. When the payload finishes its execution and attempts to restore the original execution flow, the application will blindly continue operating using the corrupted register values. Therefore, to guarantee process stability, the original CPU state must be flawlessly preserved by the instrumentation framework.

2.4 Inline Hooking

Hooking is the specific mechanism used to instrument a function. A hook allows the redirection of the execution flow from the original code to newly injected code. A standard inline hook consists of three primary logical components:

1. *The Target:* the original function whose execution we wish to intercept. The goal is to patch the first 5 to 7 bytes of the target function with an unconditional jump (`JMP`) to a user-defined payload. These initial bytes are restored to the target function once the instrumentation is complete and the framework detaches from the host process.

2. *The Detour (or Payload):* the custom function injected by the instrumentation framework. This function receives the execution flow, inspects the target function's arguments or return value, performs the desired analysis or policy enforcement, and then dictates whether the execution should continue or be blocked.

3. *The Trampoline:* in the paradigm of inline hooking, placing an interception branch (`JMP`) inherently corrupts the target function's prologue. To maintain execution continuity, all inline instrumentation frameworks must implement an Out-of-Line Execution Bridge, universally referred to as a *"Trampoline"*. Conceptually, the trampoline is a dynamically allocated, executable memory buffer that must fulfills two architectural mandates: it securely relocates the displaced prologue instructions and unconditionally routes the instruction pointer back to the unpatched remainder of the target function.

2.4.1 Signature-Enforced Hooking: Microsoft Detours and MinHook

Frameworks like Microsoft Detours [HB99] and MinHook [Kag26] are the well-known standard for Windows x86/x64. In this models, the hooking library has zero awareness of the CPU state at runtime. Consequently, they strictly enforce the application binary interface (ABI). The developer must possess precise knowledge of the target C/C++ signatures (parameter types, argument order, and return type) and calling conventions (particularly on 32-bit x86 architectures where multiple calling conventions coexist). Using these two frameworks requires a strict type definition of the target function pointer to avoid stack corruption and register mismatches, as illustrated in code 2.2.

Code 2.2: Detours and MinHook target function pointer declarations.

```
// Strict function pointer definition required by Detours
static int (WINAPI * TrueMessageBoxW)(HWND, LPCWSTR, LPCWSTR, UINT) = MessageBoxW;

// Strict function pointer definition required by MinHook
typedef int (WINAPI * MESSAGEBOXW)(HWND, LPCWSTR, LPCWSTR, UINT);
MESSAGEBOXW fpMessageBoxW = NULL;
```

Furthermore, to correctly access arguments and clean up the stack according to the ABI, the detour function must strictly mirror the original signature, as shown in code 2.3.

Code 2.3: Strict ABI compliance in the hooked detour function.

```
// Detours (same for MinHook)
int WINAPI HookedMessageBoxW(HWND hWnd, LPCWSTR lpText, LPCWSTR lpCaption, UINT uType
  )
```

The code structure for the hooked function is identical in both frameworks.

In code 2.4, the goal is to intercept a target process's call to `MessageBoxW` and modify the displayed dialog with custom text (`customCaption`) and altered options (`modifiedType`) during the PRE-LOGIC phase.

After this, the trampoline is called, the original function is executed with the modified arguments, and control is returned to the hooked version (POST-LOGIC).

Here, we intercept the user's choice (`result`), forcing a "yes" outcome even if the user pressed "no", and return the final result to the target process:

Code 2.4: Detours/MinHook Hooked function structure

```
// Detours (same for MinHook)
int WINAPI HookedMessageBoxW(HWND hWnd, LPCWSTR lpText, LPCWSTR lpCaption, UINT uType
  )
{
```

```
    // PRE-LOGIC
    UINT modifiedType = MB_YESNO;
    LPCWSTR customCaption = L"Hooked! Please choose:";

    // Call to the trampoline
    int result = TrueMessageBoxW(hWnd, lpText, customCaption, modifiedType);
    // POST-LOGIC
    if (result == IDNO)
    {
      std::cout << "[HOOK POST-LOGIC] Intercepted IDNO (No). Changing result to IDYES (
    Yes)!"
      << std::endl;
      result = IDYES;
    }
    else if (result == IDYES)
    {
      std::cout << "[HOOK POST-LOGIC] Intercepted IDYES (Yes). Nothing to do."
      << std::endl;
    }

    return result;
  }
```

2.4.2 Context-Aware Hooking: Frida-Gum

Frida-Gum [Fri26] is the underlying low-level C library of the Frida instrumentation toolkit, representing a modern, multi-architecture approach to DBI (supporting x86/x64, ARM/ARM64, MIPS).

At the core of Frida-Gum engine is a centralized execution dispatcher: when an intercepted target function is invoked, execution jumps not directly to the detour function, but into a dynamically generated entry thunk. The entry thunk saves the CPU state into an architecture-agnostic `GumCpuContext` structure and trigger the dispatcher, that automatically handle the entire hook execution flow.

Unlike traditional single-paradigm frameworks like Detours or MinHook, Frida-Gum provides a flexible, dual-paradigm Interceptor API comprising *Invocation Listener Mode* and *Function Replacement Mode*

2.4.2.1 Invocation Listener Mode

Using this mode, the developer only needs to specify the `on_enter` (injected logic before the original function) and `on_leave` (injected logic after the original function), without

worrying about defining strict C++ function signatures, since Frida-Gum wraps the captured `GumCpuContext` inside a high-level `GumInvocationContext` handle passed to user callbacks. In this way arguments and return values can be accessed and modified dynamically by their index, regardless of the target process architecture.

Due to these architectural differences, the implementation significantly diverges from that of Detours and MinHook. The Frida-Gum Invocation Listener implementation for intercepting `MessageBoxW` is illustrated in code 2.5.

Code 2.5: Frida-Gum Invocation Listener mode (`on_enter`/`on_leave`) for `MessageBoxW`.

```
// Pre-execution callback
static void
example_listener_on_enter (GumInvocationContext * ic, gpointer user_data)
{
  // Arguments accessed by index: 0:hWnd, 1:lpText, 2:lpCaption, 3:uType
  guint originalType = GPOINTER_TO_UINT (gum_invocation_context_get_nth_argument (ic,
   3));
  guint modifiedType = MB_YESNO;
  static const wchar_t* customCaption = L"Hooked, choose:";

  gum_invocation_context_replace_nth_argument (ic, 2, (gpointer) customCaption);
  gum_invocation_context_replace_nth_argument (ic, 3, GUINT_TO_POINTER (modifiedType)
  );
}

// Post-execution callback
static void
example_listener_on_leave (GumInvocationContext * ic, gpointer user_data)
{
  int result = GPOINTER_TO_INT (gum_invocation_context_get_return_value (ic));
  if (result == IDNO)
  {
    g_print ("Intercepted IDNO. Changing result to IDYES\n");
    gum_invocation_context_replace_return_value (ic, GINT_TO_POINTER (IDYES));
  }
}
```

While Invocation Listener Mode offers architecture abstraction, it introduces an inherent limitation under specific calling conventions (such as the Windows x64 ABI): since the API for accessing arguments (`gum_invocation_context_get_nth_argument`) index exclusively into General-Purpose Registers (`RCX`, `RDX`, `R8`, `R9`), parameter extraction fails for functions receiving floating-point values, which are passed via vector registers (`XMM0`–`XMM3`).

2.4.2.2 Function Replacement Mode

To overcome the limitations of index-based parameter extraction and achieve parity with traditional frameworks, Frida-Gum exposes *Function Replacement Mode* via `gum_interceptor_replace` and `gum_interceptor_replace_fast`.

1. **Standard Replacement (`gum_interceptor_replace`):** Execution passes through Frida-Gum's `enter_thunk`, where `GumCpuContext` captures the CPU state and triggers the dispatcher (identical to Invocation Listener Mode). However, instead of invoking high-level callback listeners and then jumping to the original function, the dispatcher redirects execution to a user-defined replacement function—acting as a standard C/C++ detour payload similar to Detours or MinHook (also here strict ABI compliance must be ensured). Crucially, all CPU registers (including `XMM0`–`XMM7`) are restored prior to entering the replacement function, allowing the C/C++ compiler to handle float parameters passing natively via standard C signatures.
2. **Fast Replacement (`gum_interceptor_replace_fast`):** Emits a direct assembly `JMP` instruction from the target prologue straight to the replacement function, bypassing Frida-Gum's dispatcher and `GumCpuContext` creation entirely, yielding an execution flow identical to traditional frameworks like Detours and MinHook.

The equivalent Function Replacement implementation for `MessageBoxW` is illustrated in code 2.6.

Code 2.6: Frida-Gum Function Replacement mode using native C/C++ prototypes.

```
// Function pointer type definition matching target signature
typedef int (WINAPI * TrueMessageBoxW_t)(HWND hWnd, LPCWSTR lpText, LPCWSTR lpCaption
  , UINT uType);
static TrueMessageBoxW_t TrueMessageBoxW = NULL;

// C++ Detour Function
int WINAPI HookedMessageBoxW(HWND hWnd, LPCWSTR lpText, LPCWSTR lpCaption, UINT uType
  )
{
  static const wchar_t* customCaption = L"Hooked, choose:";
  UINT modifiedType = MB_YESNO;

  // Call original implementation via trampoline
  int result = TrueMessageBoxW(hWnd, lpText, customCaption, modifiedType);

  if (result == IDNO)
    result = IDYES;

  return result;
```

```
}

// Attachment via Standard Replacement Mode
gum_interceptor_replace(interceptor,
  GSIZE_TO_POINTER(gum_module_find_export_by_name(NULL, "MessageBoxW")),
  (gpointer)HookedMessageBoxW,
  NULL,
  (gpointer*)&TrueMessageBoxW);

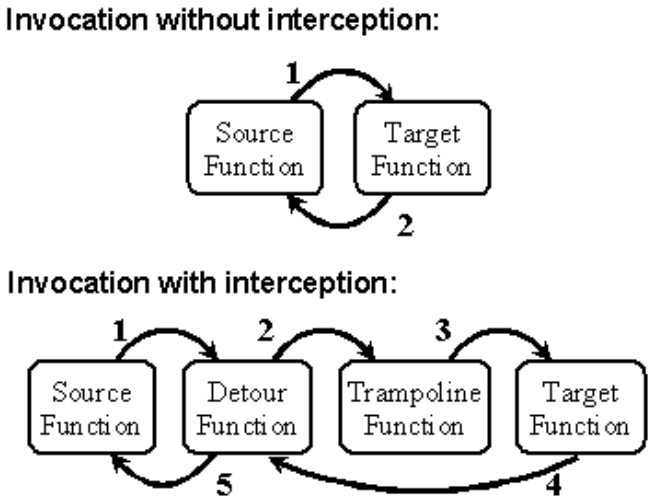
// or Attachment via Fast Replacement Mode
gum_interceptor_replace_fast(interceptor,
  GSIZE_TO_POINTER(gum_module_find_export_by_name(NULL, "MessageBoxW")),
  (gpointer)HookedMessageBoxW,
  (gpointer*)&TrueMessageBoxW);
```

2.4.3 Hook Execution Flow

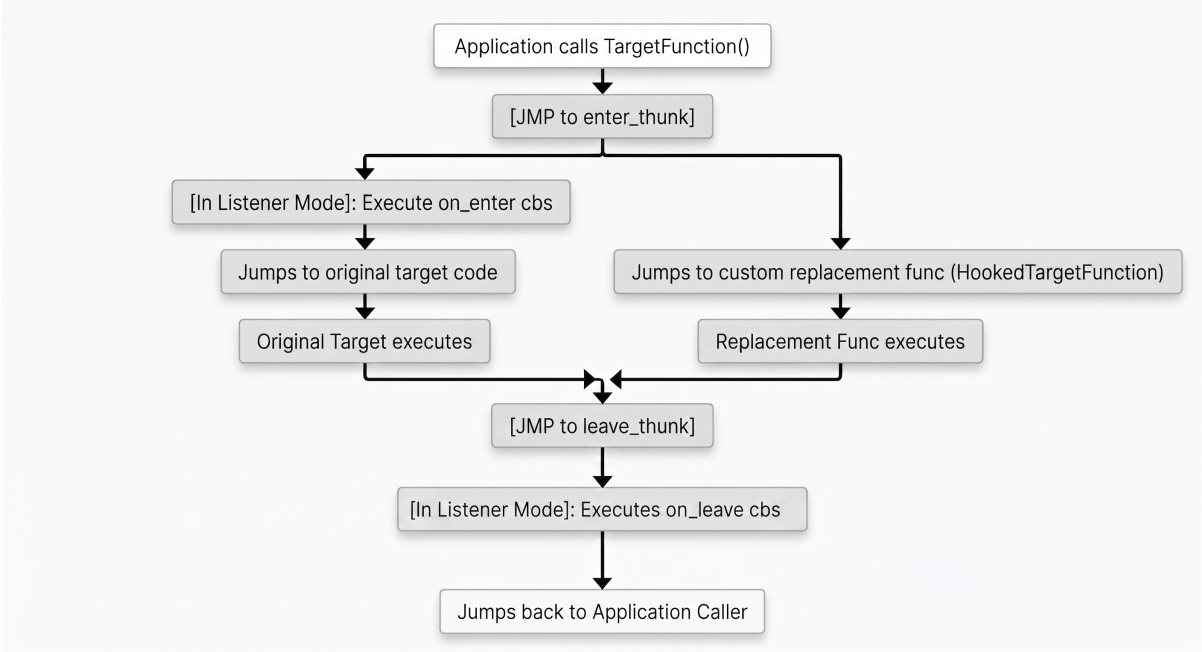
While all the presented frameworks utilize inline prologue patching, the way in which engines dispatch hook execution flow differs:

- **Passive Dispatch (Detours, MinHook, Frida Fast Replacement):** Overwrites the target prologue with a direct assembly jump (`JMP`) to the injected payload. The engine acts purely as a passive observer, delegating all execution routing and state management entirely to the user-written detour function.
- **Active Dispatch (Frida Invocation Listener Mode):** Execution jumps into a framework-managed dispatcher. It automatically dispatchs CPU register serialization (`GumCpuContext`), and optional user-defined callbacks before transferring control to the target function.
- **Hybrid Dispatch (Frida Standard Replacement Mode):** Combines active dispatcher with user-controlled execution routing. The engine intercepts execution via its active dispatcher, but then yields execution control to a user-defined replacement function. The replacement function acts as an autonomous detour function, with the same structure seen in passive dispatch frameworks.

To visually illustrate these operational differences, Figure 2.1 contrasts the direct, user-managed routing (Figure 2.1a) with the dispatcher-mediated execution path (Figure 2.1b).



(a) Detour/MinHook/Frida Fast Replacement Execution Flow (Passive)



(b) Frida Dispatcher-Mediated Flow (Active Listener / Hybrid Replacement)

Figure 2.1: A comparison of the different Hook execution flow

2.4.3.1 Advantages of Engine-Mediated Dispatch

Frida-Gum employs a framework dispatcher in both Invocation Listener (active) and Standard Replacement (hybrid) modes to deliver useful capabilities that traditional frameworks do not automatically provide:

1. **Multi-Listener Hook (Active dispatch):** With Detours or MinHook, developers are forced to manage fragile, nested chains of function pointers when multiple hooks target the same function. In contrast, Frida-Gum's active dispatcher maintains an internal list of subscribers (`listener_entries`) inside target related internal structure `GumFunctionContext`. The active dispatcher sequentially invokes the `on_enter` and `on_leave` callbacks of all attached listeners (e.g., logging, security monitoring) in a single execution pass without requiring individual listeners to be aware of one another.

2. **Built-in Thread Tracking for Safe Detachment (`gum_interceptor_flush`):** The active dispatcher automatically maintains an internal thread counter inside `GumFunctionContext`, incrementing it upon entry (`enter_thunk`) and decrementing it upon completion (`leave_thunk`). This provides native support for safe framework detachment: calling `gum_interceptor_flush()` blocks until this counter reaches zero, verifying that no worker threads remain inside hook logic before the module is unmapped.

3. **Automatic CPU State Preservation:** In traditional hooking, register preservation relies entirely on compiler-generated instructions matching the exact target ABI; any convention mismatch risks register corruption and crashes. In contrast, Frida-Gum's active dispatcher automatically serializes all CPU hardware registers into an architecture-agnostic `GumCpuContext` structure upon entry and restores them upon exit. Exposed to callbacks via `gum_invocation_context_get_cpu_context(ic)`, this mechanism guarantees process state integrity across supported architectures while allowing developers to safely inspect or modify General-Purpose Registers dynamically at runtime.

2.4.3.2 Architectural Trade-offs and the Fast Replacement Bypass

Despite their capabilities, active and hybrid dispatch introduce a non-trivial performance trade-off. Execution through the framework's centralized dispatcher (`enter_thunk`) requires allocating memory for context structures and executing multiple CPU register serializations on every function invocation. For high-frequency target routines executed millions of times per second (e.g., memory allocations or critical section acquisitions), this additional CPU overhead can degrade execution speed compared to passive jumps.

This latency penalty explains the primary role of Frida-Gum's *Fast Replacement Mode* (`gum_interceptor_replace_fast`). By emitting a raw assembly `JMP` instruction directly from the target prologue to the replacement payload, fast replacement bypasses the `enter_thunk` dispatcher and `GumCpuContext` creation entirely. This design trades listener chaining and context serialization for minimal hook latency.

2.4.4 Instruction Boundaries and the Role of Disassemblers

Advisor: Non so se mettere questa sezione

A fundamental challenge in implementing the hook structure described above, particularly on Intel x86 and x64 architectures, is the variable-length nature of the instruction set. To detour a target function, an inline hooking framework typically needs to overwrite the first few bytes of the function's prologue with an unconditional jump (`JMP`) to the custom payload. In a standard 32-bit architecture, this jump requires 5 bytes (1 byte for the opcode, 4 bytes for the relative address). If the framework blindly overwrites exactly 5 bytes, it risks cutting an existing instruction in half. If the original prologue consists of two 3-byte instructions, overwriting 5 bytes will corrupt the second instruction, leaving a hanging, invalid byte. When the trampoline attempts to execute these copied bytes to resume the program, the CPU will encounter an invalid opcode and crash the process. To prevent this, hooking frameworks must incorporate a Length Disassembler Engine (LDE). Before applying any patches, the engine disassembles the target function's prologue to calculate exact instruction boundaries. It then copies whole, unfragmented instructions into the trampoline until it has safely relocated at least the number of bytes required for the jump (padding any remaining bytes in the original function with `NOP` instructions). The choice of disassembler heavily impacts the footprint and dependencies of the framework:

- *Microsoft Detours* relies on a proprietary, internal instruction decoder optimized specifically for standard compiler prologues.
- *MinHook* utilizes a highly optimized, custom-built Hacker Disassembler Engine (HDE32/HDE64) to calculate instruction lengths, rather than fully analyze instruction semantics.
- *Frida-Gum* requires deep semantic analysis of the instructions to achieve its context-aware routing. To accomplish this, it statically links against Capstone, a comprehensive and heavy multi-architecture disassembly framework, significantly increasing the size of the resulting DLL.

2.4.5 Transactional Patching

A critical challenge in inline hooking is applying the memory patch (the `JMP` instruction) to a live, executing process without inducing a *race condition*.

In a multi-threaded application, if a worker thread attempts to execute a target function's prologue at the exact instant the instrumentation framework is overwriting its bytes, the CPU may decode a partially written, invalid opcode, triggering an immediate process crash. To prevent this, instrumentation frameworks implement a *transactional model*. However, the concrete mechanism used to guarantee concurrency safety diverges significantly depending on the design of the engine. The presented frameworks fall into two paradigms: *Pessimistic Stop-the-World Relocation* and *Optimistic Non-Blocking Batching*.

2.4.5.1 Pessimistic Stop-the-World: Thread Enlistment and IP Relocation

Frameworks tailored natively for Windows, such as *Microsoft Detours* and *MinHook*, follow a pessimistic synchronization model. Before modifying target memory, the framework temporarily suspends all concurrent worker threads in the process.

Crucially, simply suspending threads is insufficient on its own. A worker thread might have been preempted at the exact moment its Instruction Pointer (instruction pointer (IP), i.e., `EIP`/`RIP`) was positioned within the byte range of the prologue being overwritten. If resumed naively, the thread would execute the newly written destination address of the detour jump as raw machine code, corrupting execution. To prevent this, the framework inspects the register context of each thread (`GetThreadContext`). If a thread's IP falls within the replaced address range, the framework dynamically adjusts the IP to point to the corresponding relocated instruction inside the trampoline (when attaching) or back to the restored prologue bytes (when detaching).

Microsoft Detours makes this transaction explicitly developer-managed. The developer begins a transaction, explicitly enumerates and enlists every thread via `DetourUpdateThread`, queues the hook attachments or detachments, and commits the transaction, as illustrated in code 2.7. During `DetourTransactionCommit`, Detours suspends the enlisted threads, verifies and relocates their instruction pointers, applies the patches, flushes the CPU instruction cache, and resumes execution.

Code 2.7: Microsoft Detours transactional model with explicit thread enlistment and IP relocation.

```
// ATTACHMENT

// Start the transaction
DetourTransactionBegin();
```

```
// Enlist threads whose instruction pointers must be inspected and relocated
DetourUpdateThread(GetCurrentThread());
for (HANDLE hThread : workerThreads) {
    DetourUpdateThread(hThread);
}

// Queue the hook attachments
DetourAttach(&(PVOID&)TrueMessageBoxW, HookedMessageBoxW);

// Atomically suspend enlisted threads, rewrite IPs, patch bytes, and resume
DetourTransactionCommit();

// DETACHMENT
DetourTransactionBegin();

DetourUpdateThread(GetCurrentThread());
// here we specify possible other threads we want to be inspected...

DetourDetach(&(PVOID&)TrueMessageBoxW, HookedMessageBoxW);
// here we specify possible other hooks we want to detach...

// Atomically suspend threads, restore prologue bytes, fixup IPs, and resume
DetourTransactionCommit();
```

MinHook adopts the exact same underlying mechanism but abstracts the transaction boundaries from the public API (code 2.8). Internally, when `MH_EnableHook` or `MH_DisableHook` is invoked, the library enumerates process threads via a Toolhelp32 snapshot, suspends them, performs instruction-pointer relocation, applies the patch, and resumes the threads.

Code 2.8: MinHook internal thread synchronization during enable/disable phases.

```
// ---  ATTACHMENT ---
MH_CreateHook(&MessageBoxW, &HookedMessageBoxW, reinterpret_cast<LPVOID*>(&
  fpMessageBoxW));

// MH_EnableHook internally captures snapshots, suspends threads, rewrites IPs, and
  resumes
MH_EnableHook(&MessageBoxW);
//or MH_EnableHook(MH_ALL_HOOKS); if we have more than one hook

// ---  DETACHMENT ---
// MH_DisableHook internally suspends threads, restores original bytes, rewrites IPs,
   and resumes
MH_DisableHook(&MessageBoxW);
//or MH_DisableHook(MH_ALL_HOOKS); if we have more than one hook
```

2.4.5.2 Optimistic Non-Blocking Batching: Frida-Gum Transactions

In contrast to Detours and MinHook, *Frida-Gum* was designed as a cross-platform instrumentation runtime (supporting Windows, Linux, macOS, Android, and iOS). In cross-platform environments, suspending all running threads across a process is either not exposed by operating system primitives or heavily restricted by sandboxing.

Consequently, Frida-Gum does *not* implement thread enlisting, process-wide thread suspension, or register context inspection during hook installation. Instead, Frida-Gum's transactional API (`gum_interceptor_begin_transaction` and `gum_interceptor_end_transaction`) serves as a *staging and batching boundary*, as shown in code 2.9:

Code 2.9: Frida-Gum transactional lifecycle for staged, batched function replacement.

```
// ---  ATTACHMENT ---
gum_interceptor_begin_transaction(interceptor);

// Function Replacement Mode
gum_interceptor_replace(interceptor, target_func, (gpointer)HookedFunc, NULL, (
  gpointer*)&TrueFunc);
// OR
gum_interceptor_replace_fast(...)

// OR Invocation Listener Mode
// gum_interceptor_attach(interceptor, target_func, listener, NULL,
  GUM_ATTACH_FLAGS_NONE);

gum_interceptor_end_transaction(interceptor);

// ---  DETACHMENT ---
gum_interceptor_begin_transaction(interceptor);

// Revert target prologue bytes
gum_interceptor_revert(interceptor, target_func);

// OR Invocation Listener Mode
// gum_interceptor_detach(interceptor, listener);

gum_interceptor_end_transaction(interceptor);
```

2.4.6 Thread-Safe Ejection

During the *detachment phase*, restoring the target function's original prologue bytes prevents *new* executions from entering the hook payload. However, transactional byte restoration alone is insufficient to guarantee safe DLL ejection (`FreeLibrary`). In multi-threaded

applications, worker threads may already be executing inside the custom detour logic when prologue reversion occurs. If the host process unmaps the instrumentation DLL memory while a worker thread is active inside the hook logic, the thread encounters unmapped virtual memory, triggering an immediate Access Violation (`0xC0000005`).

To achieve thread-safe ejection, inline hooking architectures must supplement transactional byte restoration with a *Synchronization mechanism*. As illustrated in code 2.10, two distinct approaches can be used to achieve safety ejection:

1. **Manual Tracking:** Because Detours and MinHook have no internal thread awareness, the instrumentation wrapper must wrap each detour function in a guard that atomically increments a global counter upon entry and decrements it upon exit. During detachment, after committing the prologue restoration transaction, the payload spin-waits until the counter reaches zero.

2. **Built-in Tracking:** Because Frida-Gum's active dispatcher automatically tracks active threads across `enter_thunk` and `leave_thunk`, it exposes a native flushing API. During detachment, after reverting patches, `gum_interceptor_flush()` is called in a spin-wait loop, it returns `TRUE` only when all in-flight threads have completely finished.

Code 2.10: synchronization protocols for thread-safe module ejection in Detours and Frida-Gum.

```
// ========================================================================
// Detours / MinHook
// ========================================================================
static std::atomic<int> g_activeHooks{0};
struct HookGuard {
    HookGuard()  { g_activeHooks++; }
    ~HookGuard() { g_activeHooks--; }
};
void HookedProcessTask(int taskId, const char* taskType, int complexity) {
    HookGuard guard; // Automatically tracks thread execution scope
    // Custom detour logic...
    TrueProcessTask(taskId, taskType, complexity);
}
void SafeDetach_Detours() {
    // 1: Revert prologue bytes via transactional patch
    DetourTransactionBegin();
    // ...

    // 2:  wait for in-flight threads to exit
    while (g_activeHooks.load() > 0) {
        SwitchToThread();
    }
```

```
}
// ============================================================================
// Frida-Gum
// ============================================================================
void SafeDetach_Frida(GumInterceptor* interceptor, gpointer target_func) {
    // 1: Revert prologue bytes via transactional patch
    gum_interceptor_begin_transaction(interceptor);
    // ...

    // 2: wait for active dispatcher signal to flush
    while (!gum_interceptor_flush(interceptor)) {
        SwitchToThread();
    }
}
```

2.4.7 Summary of Inline Hooking Frameworks

To conclude the discussion on inline hooking, Table 2.1 provides a summary comparison of the three analyzed frameworks: Microsoft Detours, MinHook, and Frida-Gum.

Feature	**Detours**	**MinHook**	**Frida-Gum**
CPU State Preservation	Manual	Manual	Automatic (`listener`) / Manual (`replace`)
Hook Dispatch	Passive	Passive	Active (`listener`) / Hybrid (`replace`)
ABI Constraints	Strict	Strict	Abstracted (`listener`) / Strict (`replace`)
Disassembler	Proprietary LDE	HDE32 / HDE64	Capstone (full semantic decoder)
Patching Concurrency	Stop-the-World (explicit thread enlistment & IP relocation)	Stop-the-World (automated thread suspension & IP relocation)	Non-blocking batching (atomic instruction writes, no thread enlistment)
Architectures	x86, x64	x86, x64	x86, x64, ARM, ARM64, MIPS

Table 2.1: Comparison of the analyzed inline hooking frameworks.

2.5 Code-Cache-Based DBI Architectures

For deeper execution analysis at the assembly level, code-cache-based DBI engines are utilized. Unlike inline hooking, which patches the original binary in place, code-cache-based DBI engines ensure that the target application's original code is never executed directly by the CPU. Instead, the framework employs a *Just-In-Time (JIT) compilation* mechanism to dynamically translate and instrument instruction sequences.

2.5.1 Core Architectural Components

Modern code-cache-based DBI frameworks rely on a unified architectural pattern comprising the following components:

1. **The Dispatcher (Control Flow Manager):** The dispatcher is the central component of the code-cache DBI engine, responsible for orchestrating the execution state. Because the application and the code-cache DBI framework share the same process but operate under different scopes, they must safely transit between "framework mode" and "application mode". The dispatcher triggers a context-switch mechanism that carefully saves and restores the application's state whenever execution exits or enters the code cache.

2. **Just in time (JIT) Compiler:** Instead of translating the entire binary before execution, these architectures translate the application's instructions dynamically in small fragments. During this translation phase, the framework decodes and encodes (into the framework's abstraction) the original instructions and has the opportunity to insert custom instrumentation code or patches before emitting the new sequence.

3. **The Code Cache (The Execution Sandbox):** To mitigate the massive performance penalty of repeatedly translating the same code, code-cache-based DBI frameworks employ a software-based code cache. The translated and potentially instrumented fragments are stored in this sandboxed memory region. When the application re-executes a previously translated section of code, it runs directly from the code cache, which significantly optimizes execution speed.

4. **The Client Interface (Instrumentation API):** This is the abstraction layer exposed to the developer. It provides the necessary functions to inspect the program state, traverse instruction sequences, and specify exactly where and how instrumentation should be injected.

2.5.2 The Execution Lifecycle: The JIT Loop

To formally describe how these components interact at runtime, we can outline the execution lifecycle of a standard block of code, illustrated in Figure 2.2:

1. *Interception:* The target application attempts to execute a block of original code. The code-cache DBI engine intercepts this attempt and yields control to the Dispatcher.

2. *Cache Lookup:* The Dispatcher queries a fast-lookup hash table to check if the target memory address has already been translated. If it results in a cache hit, the CPU is redirected to the Code Cache.

3. *Translation (On Cache Miss):* If the code has not been encountered before, the JIT Compiler decodes original binary instructions one by one to form a straight-line sequence that terminates at a control transfer, such as a branch, call, or return.

4. *Instrumentation:* The engine queries the Client Interface to determine if the user wishes to instrument the block or a single instruction within it. If so, the instrumentation instructions are injected.

5. *Emission and Linking:* Most code-cache-based DBI frameworks use this phase to apply patches to the control-flow instructions at the end of the sequence, guaranteeing that when the block finishes running, it will redirect back to the framework rather than jumping to a native memory address. If the block ends in a direct branch to another previously translated block, the engine "links" them together directly in the cache (if it was an indirect branch, it cannot know the target before executing it), allowing the CPU to bypass the Dispatcher in future iterations. The modified block is emitted into the Code Cache.

6. *Execution:* The Dispatcher performs a context switch, restoring the application state, and commands the CPU to execute the newly cached block natively.

7. *Code Cache Exit:* Execution remains safely inside the code cache — often jumping directly from one translated block to another through "linking"—until it hits a boundary. This boundary might be an unlinked branch or an indirect branch whose target hasn't been translated yet. When this happens, the block executes an exit stub which saves the CPU's state and transfers control back to the Dispatcher, restarting the loop.

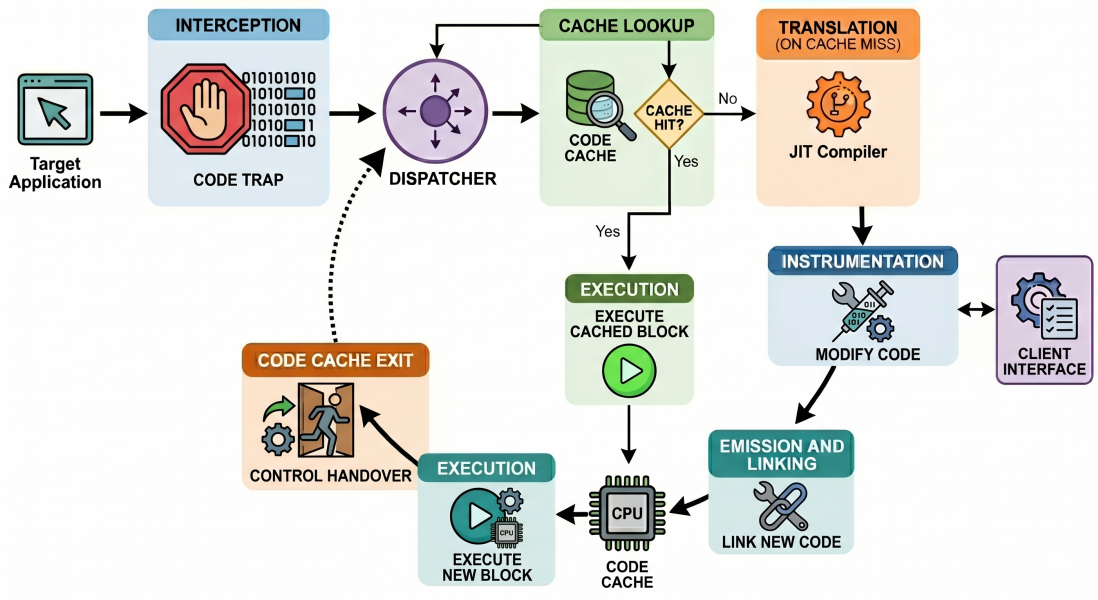


Figure 2.2: The Execution Lifecycle of a Code-Cache-Based DBI framework

While code-cache-based DBI frameworks allow a more granular analysis of the target code, they introduce *significantly higher overhead* compared to inline hooking. Every single instruction of the program needs to be JIT-compiled before execution, even if no instrumentation is injected.

2.5.3 The Client Interface: Instrumentation vs. Analysis Functions

Code-cache-based DBI frameworks require the developer to interact with the engine using a *two-tiered callback paradigm*. The developer must supply two distinct types of functions to the engine, each operating at fundamentally different times during the execution lifecycle:

- *The Instrumentation Function (The "Where" and "What"):* This callback is invoked by the code-cache DBI engine *only once* per newly discovered sequence of instructions during the JIT compilation phase. Its purpose is to inspect the static properties of the instructions (e.g., opcodes, operands) and decide *if* and *where* custom code should be injected. If injection is required, it uses the API to queue the insertion of the analysis function.
- *The Analysis Function (The "Action"):* This is the actual custom payload injected into the Code Cache. Unlike the instrumentation function (executed only at compile

time), it is executed *every time* the CPU runs the corresponding instrumented block of code. For this reason, the analysis function must be heavily optimized and efficient.

The level of abstraction provided by the Client Interface significantly alters how developers interact with the engine.

2.5.4 High-Level Abstraction API: Intel PIN

Intel PIN [Luk+05] is a standard academic code-cache-based DBI framework with a proprietary (free for non-commercial use) license. It provides a heavily abstracted C++ API.

To utilize Intel PIN, developers do not modify the core engine itself. Instead, they write a custom C++ program known as a *PINtool*. A PINtool represents the physical implementation of the Client Interface discussed previously; it is the user-defined module that contains both the Instrumentation Functions and the Analysis Functions. Architecturally, a PINtool is compiled as a standard shared library (a DLL on Windows). When the user launches a target application under PIN, the engine automatically injects the PINtool DLL into the target process's memory space. Once loaded, the engine invokes the PINtool's `main` function, allowing the tool to register its specific callbacks. In code 2.11, we can observe the granularity achievable using code-cache-based DBI: in this case, we can log conditional branches to determine which paths are taken during code execution.

Code 2.11: An example of a PINtool for branch logging

```
// Analysis function (Runs at execution time in the Code Cache)
void record_branch(ADDRINT instruction_address, ADDRINT target_address, BOOL taken) {

  if (taken) { /* Log Taken */ }
  else       { /* Log Not Taken */ }
}

// Instrumentation function (Runs at JIT compilation time)
void trace(INS instruction, void *v) {

  // PIN abstracts away module resolution: we want to instrument
  // only target application instructions, filter possible external libraries
  RTN rtn = INS_Rtn(instruction);
  if (RTN_Valid(rtn) && IMG_IsMainExecutable(SEC_Img(RTN_Sec(rtn)))) {
    // If it is inside the target executable, check if it's an IF branch
    if (INS_IsBranch(instruction) && INS_HasFallThrough(instruction)) {
      // Inject analysis function right before the jump happens.
      INS_InsertCall(instruction, IPOINT_BEFORE, (AFUNPTR)record_branch,
      IARG_INST_PTR,
      IARG_BRANCH_TARGET_ADDR,
      IARG_BRANCH_TAKEN,
```

```
        IARG_END);
      }
    }
  }
```

2.5.5 low-level API: DynamoRIO

DynamoRIO [BGA03] is the open-source alternative that exposes a lower-level C API. The equivalent branch-logging example seen previously for PIN is implemented in code 2.12.

Code 2.12: DynamoRIO branch instrumentation example

```
// Analysis function
static void record_branch(app_pc instr_addr, app_pc target_addr, int taken) {

  if (taken) { /* Log Taken */ }
  else       { /* Log Not Taken */ }
}

// Instrumentation function
static dr_emit_flags_t event_bb_analysis(void *drcontext, void *tag, instrlist_t *bb,
   instr_t *inst, bool for_trace, bool translating,
void *user_data) {

  // Manual module tracking to filter out OS libraries
  app_pc instr_addr = instr_get_app_pc(inst);
  module_data_t *mod = dr_lookup_module(instr_addr);
  bool is_main_exec = false;

  if (mod != NULL) {
    module_data_t *main_mod = dr_get_main_module();
    if (main_mod != NULL) {
      if (mod->start == main_mod->start) { is_main_exec = true;
      }
      dr_free_module_data(main_mod);
    }
    dr_free_module_data(mod);
  }

  if (!is_main_exec) return DR_EMIT_DEFAULT;
  // Check if it's a conditional branch (CBR)
  if (instr_is_cbr(inst)) {
    // Inject analysis function into the basic block (bb)
    dr_insert_cbr_instrumentation(drcontext, bb, inst, (void *)record_branch);
  }
  return DR_EMIT_DEFAULT;
}
```

Using DynamoRIO, developers can directly manipulate the instruction flow. In PIN, developers generally "wrap" existing instructions (adding a call before or after an instruction) but cannot easily modify the instruction itself. In DynamoRIO, the developer is handed a linked list of instructions (`instrlist_t`) and can easily manipulate or modify them (e.g., reordering them or moving a load instruction earlier to test for race condition).

Furthermore, because PIN is designed to be highly opaque, using `INS_InsertCall` prompts PIN's JIT compiler to automatically analyze the custom analysis function for deciding which registers need to be saved and restored based on what the C++ code actually uses. With DynamoRIO, developers can manage the state and flags manually for specialized analysis.

In conclusion, while DynamoRIO may present a steeper learning curve than Intel PIN, it affords developers greater transparency and raw power, providing fine-grained control over the translated code and the code cache.

2.5.6 Summary of Code-Cache-Based DBI Frameworks

To summarize the key differences between the analyzed code-cache-based DBI engines, Table 2.2 compares the features and capabilities of Intel PIN and DynamoRIO.

Feature	**Intel PIN**	**DynamoRIO**
API Style	High-level C++ API	Low-level C API
Code Modification	High-level wrapping (instrumentation only before/after instructions)	Direct instruction manipulation
State/Flags Handling	Fully automatic (always analyzed and optimized by JIT compiler)	Manual or automatic (developer can choose)
Platform Support	Windows, Linux, macOS (mainly x86/x64)	Windows, Linux, macOS, Android (x86, x64, ARM, ARM64)
License	Proprietary (Free for non-commercial use)	BSD (Open Source)

Table 2.2: Comparison of the analyzed code-cache-based DBI frameworks.

2.6 Comparison of Approaches

To conclude the overview of the DBI ecosystem, this section provides a high-level comparison of the two paradigms discussed in this chapter: Inline Hooking and Code-Cache-Based DBI. Table 2.3 highlights the fundamental differences between these two methodologies.

Dimension	**Inline API Hooking**	**Code-Cache-Based DBI**
Basic Mechanism	Overwrites instructions in-place (e.g., using a `JMP` in the prologue).	Translates and executes instructions in a sandboxed Code Cache via JIT compilation
Granularity	Function-level (entry and exit points).	Assembly Instruction-level.
Performance Overhead	Negligible (execution of detour and trampoline after initial jump).	Non-trivial (continuous decoding, JIT compiler loop, of all instructions).
Safety & Isolation	Lower (if not ABI compliant immediately crashes the target).	Higher (sandboxed cache; register and status flags saving is automated by the engine)
Memory Integrity	Modifies original binary pages in memory (requires altering page write permissions).	Preserves original code pages intact (modifications occur only within the cache)
Complexity	Low to Moderate (relies on basic disassembler length check and prologue displacement).	High (requires a full JIT compiler, control flow manager, and basic block linker).

Table 2.3: Comparison of the two main DBI paradigms.

2.7 Experimental Validation

To empirically validate the theoretical concepts discussed in the preceding sections and to fully understand the operational constraints, security boundaries, and low-level deployment challenges of each framework, a propedeutic suite of testing applications was developed.

All source code and custom injection utilities have been organized and made available within the project's repository.[1]

The experimental activity was structured into three different scenarios, each designed to evaluate specific architectural properties of the instrumentation engines.

2.7.1 Scenario 1: Single-Threaded Hooking and Process Injection/Ejection Mechanics

- **Test Rationale:** This preliminary scenario aimed to verify the fundamental lifecycle management of DBI within the Windows ecosystem.

  This involved implementing a custom, dual-mode utility (`injector.cpp`) to handle both the insertion and the ejection of the instrumentation payload.

  Concurrently, the test evaluated the syntactic and structural paradigms of the three inline hooking frameworks (Microsoft Detours, MinHook, and Frida-Gum) when intercepting a standard system API—`MessageBoxW` in `user32.dll`.

- **Empirical Results:** Every analyzed framework successfully intercepted the execution flow during the "Pre-Logic" phase. This capability allowed for the modification of string pointers targeting the dialog box text and caption, the reassignment of the configuration flags to `MB_YESNO`, and the runtime mutation of the return value during the "Post-Logic" phase, effectively forcing an `IDYES` outcome regardless of the user's explicit interaction.

  Furthermore, forcing the programmatic module ejection successfully triggered the `DLL_PROCESS_DETACH` branch within each hook's entry point, safely restoring the original prologue instruction bytes and confirming a flawless ejection in single-threaded environments.

- **Encountered Obstacles:** In this idealized single-threaded scenario, no big challenges occurred. Setting up Microsoft Detours was straightforward due to its thoroughly documented APIs, the abundance of reference examples, and a standard paradigm that is conceptually simpler than that of Frida-Gum.

  MinHook demonstrated a similar workflow to Detours but simplified the implementation further by automatically managing transactional patching internally.

  Conversely, approaching Frida-Gum for the first time presented a steeper learning curve; given its more complex architecture, distinct paradigm, and less exhaustive documentation; comprehensively understanding its API behaviors and mapping them

[1] https://github.com/davde342/MasterThesis/tree/main/code/first_examples

to my use case required a direct analysis of the huge and dense codebase within the official Frida repository.

2.7.2 Scenario 2: Multi-Threaded Hooking and Concurrency Management

- **Test Rationale:** To simulate a realistic production workload and test the instrumentation frameworks under thread concurrency, a multithreaded target engine was used (`target_multithread.cpp`). This target utilizes a synchronized task queue where a producer thread continuously queues activities with varying workloads, while three concurrent consumer threads process these tasks in parallel by invoking the exported routines `ProcessTask` and `LogTaskResult`.

- **Empirical Results:** Real-time monitoring of thread behaviors was achieved, allowing a dynamic optimization policy: the injected hook intercepted high-latency tasks (>500ms) and programmatically constrained their complexity metrics to 100ms prior to native routing, illustrating the possibility of inline hooking for runtime policy enforcement and load adaptation.

- **Encountered Obstacles:** Two major obstacles I faced in this scenario. First, performing hook detachment in a concurrent context introduced a possible race condition: active worker threads could be descheduled while executing inside the hook callback, causing an immediate Access Violation (`0xC0000005`) once the library was unmapped.

  Second, in Frida-Gum, intercepting routines that process floating-point arguments (such as the `double` parameter in `LogTaskResult`) encountered an ABI mismatch because these parameters are passed via `XMM0`–`XMM3` registers under the Windows x64 ABI, while Frida expects them in other registries.

- **Implemented Mitigations:** To address the detachment race condition in concurrent environments, framework-specific synchronization protocols were deployed (as formalized in Section 2.4.6 and code 2.10).

  As detailed in Section 2.4.2, to bypass the floating-point register limitation inherent in Frida-Gum's Invocation Listener mode, the high-level callback attachment was discarded in favor of Frida-Gum's Function Replacement mode via `gum_interceptor_replace`, allowing the C/C++ compiler to handle exact register allocation natively.

2.7.3 Scenario 3: Instruction-Level Assembly Tracking via Code-Cache-Based DBI Engines

- **Test Rationale:** To achieve low-level instruction granularity, a target program containing conditional branches was compiled (`PIN_dynamo_target.cpp`), alongside an Intel PIN tool (`pintool.cpp`) and a DynamoRIO client (`dynamo.c`). The rationale was to program the underlying JIT compilers to isolate all conditional branch instructions (`INS_IsBranch` / `instr_is_cbr`) and dynamically inject an analysis callback (`record_branch`) used to log the target address (taken branch).
- **Empirical Results:** Both code-cache-based DBI engines yielded execution trace graphs. Using IntelPIN it was also possible to write the source file code line number correspondent to the taken branch
- **Encountered Obstacles:** An initial operational challenge involved navigating the different APIs of the two frameworks; DynamoRIO exposes lower-level, low-level utilities compared to the highly abstracted API of Intel PIN, resulting in a steeper learning curve. Beyond API familiarity, the primary technical obstacle across both engines was attempting to log not only the target runtime address of each taken branch but also its source code line number from `PIN_dynamo_target.cpp` (compiled with debug symbols) to maximize execution trace readability.
- **Implemented Mitigations:** After several unsuccessful attempts, I have found that for solving the source line number obstacle under Intel Pin, I had to utilize the `PIN_GetSourceLocation` API. However, direct invocation of this routine inside the analysis function triggers a critical runtime error due to missing client locks. To mitigate this, calls to `PIN_GetSourceLocation` were wrapped inside a thread-safe synchronization pair using `PIN_LockClient()` and `PIN_UnlockClient()`, ensuring proper access to Pin's symbol database.

Chapter 3

Methodology

This chapter presents the design, architecture, and implementation of HookDesigner: a declarative, multi-target transpilation and telemetry framework for Windows DBI. The chapter is structured as follows: Section 3.1 establishes the design principles, the overall architecture, and the target hooking engines. Section 3.2 introduces the declarative policy specification. Section 3.3 describes the template-driven code generation process. Section 3.4 outlines the asynchronous logging and metrics collection subsystems. Section 3.5 examines runtime safety, crash prevention, and fault tolerance mechanisms. Finally, Section 3.6 details payload deployment, process injection, and safe module ejection.

3.1 Design Principles and Architecture

The core objective of HookDesigner is to eliminate the gap between *policy specification*—the *what* of an instrumentation policy—and *production-ready instrumentation code*—the *how* of implementing that policy against a concrete hooking engine. This gap is traditionally bridged by hand: a developer reads a policy document, then manually translates each requirement into engine-specific code. However, handcrafting instrumentation code is notoriously error-prone due to the complete virtual address space sharing between the host application and the injected payload. For this reason, subtle programming errors—such as dereferencing invalid parameters, or encountering race conditions in multi-threaded programs—inevitably result in unhandled crashes that terminate the entire host process. Moreover, managing the lifecycle of hooks, including safe runtime attachment/detachment, produces fragile, tightly coupled to a specific engine code that is difficult to audit and maintain under changing requirements.

To overcome these limitations, HookDesigner introduces a novel paradigm in DBI: *declar-*

ative, multi-target transpilation. In computer science, a *transpiler* (or source-to-source compiler / code generator) is a program that translates an input specification or program authored in one high-level language into equivalent source code in another programming language. HookDesigner embodies this approach through a transpiler that automatically translates high-level, human-readable policy declarations directly into the corresponding engine-specific C++ code.

HookDesigner targets the setting of Chapter 1: Windows industrial processes for which source code is unavailable and the process cannot be restarted at will, the deployment host is often restricted (no compiler, interpreter or installer), the application is multi-threaded and latency-sensitive, and a crash of the instrumented process is unacceptable.

Trust model: Policy authors are trusted: DSL expressions are compiled into the payload, which runs with the privileges of the target. Section 3.1.1 derives the design principles from these requirements and records, for each, the alternative that was considered.

HookDesigner addresses this challenge through five foundational design principles.

3.1.1 Design Principles

- **Engine-Independent Transpilation** As analyzed in Inline Hooking (Section 2.4), inline hooking engines expose different APIs, and constraints on the instrumentation code. Rather than hard-coding engine details, HookDesigner treats each engine as a plug-in rendering target. A YAML engine registry (`engines.yaml`) maps engine names to Jinja2 template files, library paths, and linking arguments. The transpiler renders one complete C++ translation unit per engine in a single pass, producing compilable output. Adding a new engine requires only a new template and a registry entry; the core transpiler logic remains unchanged.

- **Crash Safety** As analyzed in Section 2.2.2, the instrumentation payload is mapped directly into the target application's private virtual address space. Consequently, any runtime error or unhandled memory access violation in the hook payload will crash the entire host process. HookDesigner enforces crash safety through automated fault isolation, generating safe memory-access primitives around all parameter reads, pointer dereferences, and wrapping user callback executions and telemetry code in exception handlers.

- **Thread Safety** Hooking a running, multi-threaded process introduces synchronization issues during both hook installation/removal and module ejection (as analyzed in Section 2.4.6). HookDesigner addresses these challenges through transactional patching models during attachment and detachment, combined with automatic thread-tracking guards embedded in each hook.

- **High-Performance Telemetry** To provide observability without damaging application responsiveness, the telemetry must introduce minimal overhead on critical execution paths. HookDesigner integrates industry-standard tools for high-performance telemetry, combining asynchronous logging via `spdlog` [Mel26] with lightweight metric exposition through `Prometheus` [The26b; Mp26]. This allows payloads to collect logs and real-time performance data with negligible latency impact on the host application.
- **Separation of Policy from Code** The instrumentation policy is expressed in a declarative YAML document that specifies *which* functions are intercepted and *what* behaviors are applied for each hook (logging rules, performance metrics, pre/post callbacks). The callback implementations and the target function signatures are authored separately in a standard C++ source file using a comment-based tagging convention, giving policy authors full IDE support and syntax highlighting. This separation allows also the same logic file to be reused across different policies and different target engines.

3.1.2 The Seven-Stage Pipeline

HookDesigner processes an instrumentation request through a pipeline of seven stages. The pipeline is split into two execution contexts: an *offline* context executed on the developer's workstation (stages 1–4), and an *online* context executed inside the target process' environment after the compiled DLL has been transferred to the deployment machine (stages 5–7).

- **Stage 1 – Input:** The user provides two artifacts: a YAML policy file that declaratively specifies the instrumentation configuration, and a C++ logic file that contains the implementations of pre-hook and post-hook callbacks and the signature of the functions to instrument specified in the policy. The policy file declares the target process name, the logic file path, the selected engine(s), optional telemetry configuration, and the list of functions to hook with their associated behaviors.
- **Stage 2 – Parse & Validate:** The transpiler parses the YAML policy and validates its structural and semantic integrity against the schema, verifying consistency and cross-referencing declarations with data in the C++ logic file.
- **Stage 3 – Generate:** The validated policy data are passed to the code generation engine. Using modular templates, the transpiler generates complete, self-contained C++ source code tailored to the requested hooking engine.
- **Stage 4 – Compile:** The generated C++ source files are compiled into Windows DLLs. The build process statically links the chosen instrumentation engine and

telemetry backends, producing self-contained binaries without external runtime dependencies.

- **Stage 5 – Inject:** The compiled DLL is loaded into the target process by the injector utility (`injector.exe`, implemented in `injector.cpp`), which supports both attaching to an already-running process and launching a new process with the instrumentation payload active from its startup.
- **Stage 6 – Execute:** Inside the running target, the injected hooks wrap target function, executing custom pre- and post-execution callbacks, recording performance metrics, and evaluating logging rules.
- **Stage 7 – Eject:** When instrumentation terminates, the injector initiates a safe detachment procedure. Hook modifications are atomically reverted, and module unloading is coordinated with in-flight thread tracking to prevent process crashes.

A key consequence of the split into two execution contexts is that the online stages require no Python runtime, LLVM/LIBCLANG installation, and no access to the policy source—only the compiled DLL and the injector binary are needed on the deployment host. This is especially relevant for industrial environments, where the target host is typically a restricted system that lacks developer runtimes and on which installing external dependencies or interpreters is often blocked.

3.1.3 The HookDesigner Architecture

To link the presented pipeline to the codebase, Table 3.1 maps each component of HookDesigner to its corresponding source files and responsibilities.

Component	Source File	Responsibility
Transpiler	`transpiler.py`	Orchestrates parsing/validation, logic file validation, Jinja2 rendering, and MSVC compilation.
Engine Registry	`engines.yaml`	Define engine specific templates, static libraries, and compiler flags.
Process Injector	`injector.cpp`	Utility managing DLL injection & ejection.
Code Templates	`templates/ detours_template.cpp` `frida_template.cpp`	Modular Jinja2 C++ templates generating engine-specific instrumentation code

Table 3.1: Mapping of HookDesigner components to codebase files.

3.1.4 Selection of Compilation Targets

The architectural analysis and experimental results presented in Background (Chapter 2) directly guided the choice of instrumentation engines supported by HookDesigner.

Currently, HookDesigner is only compatible with inline hooking paradigm (Section 2.4).

- **Microsoft Detours:** Microsoft Detours was selected because, despite its minimalistic model (analyzed in Section 2.4.1), it represents the gold standard for DBI in Windows x86 and x64 environments. By delegating execution routing directly to the detour function without intermediate layers (as illustrated in Figure 2.1a), Detours achieves near-zero runtime overhead, prioritizing execution speed over framework complexity.
- **Frida-Gum:** Frida-Gum was chosen as a complementary engine to provide modern dynamic interception capabilities. Only Function Replacement mode is used, because the Invocation Listener cannot read floating-point arguments on Windows x64 (Sections 2.4.2 and 2.7.2). This allows developers to manually handle floating-point parameter extraction bypassing Frida indexing, while retaining some of Frida's features not included in Detours/MinHook (presented in Section 2.4.3.1).

[MinHook was not chosen because its execution model matches Detours]

3.2 Declarative Policy Specification and Logic File Integration

This section analyzes the two input artifacts processed by HookDesigner in Stage 1 of its pipeline (Section 3.1.2): the declarative YAML policy file, which defines *what* functions to intercept and *which* behaviors to attach, and the C++ user logic file, which implements *how* pre/post callbacks behave.

3.2.1 The Core YAML Policy DSL

The policy DSL is structured as a human-readable YAML document containing two top-level blocks: `configure`, which establishes global setting for the instrumentation, and `hooks`, which specifies the list of target functions to intercept along with their runtime address resolution strategies and optional code to inject (pre/post callbacks).

3.2.1.1 The `configure` Block

The `configure` section declares target process parameters, the relative path to the companion C++ logic file, the instrumentation paradigm and the correspondent target engines, as summarized in Table 3.2.

Field	Type	Req.	Default	Description
`process`	string	Yes	—	Target executable filename (e.g., `scada_engine.exe`).
`logic_file`	string	Yes	—	Relative path (from the policy file) to the C++ logic source file.
`engines`	string / list	No	`"detours"`	Instrumentation engine(s) to generate code for. (e.g., `["detours", "frida"]`).
`paradigm`	string	No	*inferred*	Hooking paradigm (e.g., `inline_hook`).
`telemetry`	object	No	`{}`	Global Telemetry settings (detailed in Section 3.4)

Table 3.2: Schema definition for the top-level `configure` block.

- `engines` accepts either a single engine name or a list of target engines. When multiple engines are declared, the transpiler generates separate, engine-specific translation

C++ files in a single invocation, enabling multi-engine deployments from a single policy definition.

- `paradigm` (for the moment can be only inline_hook) explicitly declares the instrumentation paradigm. If specified, transpiler validates that all `engines` belong to this paradigm. If omitted, it is automatically inferred from the configuration registered in `engines.yaml` of the specified `configure.engines`.

3.2.1.2 The `hooks` List

The `hooks` section defines a sequence of hook configuration objects. Each entry specifies a target function to intercept, the runtime address resolution strategy to locate its address within `configure.process`, and the optional behavioral policies to apply, as detailed in Table 3.3.

Field	Type	Req.	Default	Description
`function_name`	string	Yes	—	Target function name.
`module`	string	No	`""`	Target DLL module or `configure.process`.
`is_noreturn`	bool	No	`false`	Flag for non-returning functions.
`address_resolution`	object	No	`{strategy: "export"}`	Strategy for runtime function address lookup.
`behaviors`	object	No	`{}`	Sub-policies for metrics, logging, and callbacks.
Address Resolution Configuration (`address_resolution`)				
`strategy`	string	Yes	—	Resolution method: `"export"` or `"offset"`.
`symbol_name`	string	No	`function_name`	Exported decorated symbol for `GetProcAddress`.
`rva_offset`	string / int	If offset	—	Hexadecimal RVA offset (e.g., `"0x1A30"`).
Behavioral Sub-Policies Configuration (`behaviors`)				
`security_policy`	object	No	`{}`	Custom pre-hook and post-hook callback bindings.
`log_rules`	list	No	`[]`	Logging rules (detailed in Section 3.4).
`performance_metrics`	object	No	`{}`	Target latency and Prometheus metrics (detailed in Section 3.4)

Table 3.3: Schema definition for the `hooks` list entries

- **Address Resolution Strategies:**
  - *Export Strategy (`strategy: export`):* Resolves the function entry address dynamically via `GetProcAddress`. When hooking C++ member functions or overloaded APIs where compiler name mangling alters the exported symbol name, the optional `symbol_name` field allows specifying the decorated symbol.
  - *Offset Strategy* (`strategy: offset`): For unexported functions, HookDesigner computes its address at runtime by adding the specified hexadecimal RVA offset (`rva_offset`) to the base address of the specified `module`.

- **Non-Returning Functions (`is_noreturn`):** this flag is automatically inferred for standard non-returning APIs (such as `RaiseException`, `ExitProcess`, and `ExitThread`). Because these functions never return to the caller, the transpiler leverages this flag during semantic validation to prevent policy authors from defining post-trampoline callbacks, non-compatible telemetry options (Section 3.4.3) and for correctly selecting the hook function structure designed for non-returning functions Section 3.5.4.2.
- **Security_policy**: is a dictionary containing the user-defined `pre_hook` name (executed prior to the trampoline) and `post_hook` name (executed after original function completes) of the callbacks implemented in the C++ logic file.

3.2.2 Logic File Integration

A core design principle of HookDesigner is the separation between declarative policy and imperative logic to inject, as presented in Section 3.1.1.

For guiding the transpiler in parsing the logic file, it must present a specific structure built using tag markers:

1. **Signatures Block** (`// @signatures-start` ... `// @signatures-end`): This block defines the exact parameter types, parameter order, optional calling conventions, and return types of the functions targeted by the policy.
2. **Callback Implementation Regions** (`// @<CallbackName>-start`): Each user callback implementation is prefixed with a dedicated tag matching the callback name referenced in the YAML policy's `security_policy` dictionary (`pre_hook` or `post_hook`).

The logic file will be included directly via `#include` in the transpiler generated C++ file, so it can be used as a standard helper file.

3.2.2.1 Clang Parsing

Parsing C++ function declarations using naive regular expressions is notoriously fragile due to complex types, pointer nesting, and calling conventions. To guarantee correctness, Stage 2 of HookDesigner integrates Clang via its Python bindings (`clang.cindex`) [LLV26].

The transpiler uses CLANG creates an in-memory Abstract Syntax Tree correspondent to the logic file with `-std=c++17` standard and traverses its nodes for performing the following task:

1. **Extracting Signatures:** The transpiler locates the Signatures Block and iterates over all function declarations. For every signatures node within this range, it extracts:
    - The *target function name* and *return type*
    - The *calling convention* by scanning the token preceding the function identifier, looking for calling convention keywords (`__stdcall`, `__cdecl`, `__fastcall`, etc.).
    - The *parameter list*, capturing parameter names and Clang type handles (`_clang_type`).
2. **Extracting Callback Parameters:** The transpiler scans for `// @<CallbackName>-start` tag patterns and filters the `FUNCTION_DECL` nodes, extracting parameter name for cross-validation against target function prototypes.

During the parse and validation stage, the transpiler enforces the following semantic constraints:

1. **Signature Bindings:** Every `function_name` specified in the YAML policy must have a corresponding function declaration inside the Signatures Block.
2. **Callback Resolution:** Every callback identifier declared under `security_policy` (`pre_hook` - `post_hook`) must have a matching `// @<CallbackName>-start` implementation in the logic file.
3. **Parameter Matching:** Each callback parameter must match the name of a parameter in the target function's signature. Type checking and reference binding to the C++ compiler, since we could have arguments passed as reference or type aliases.
4. **Return Value Overriding:** For `post_hook` callbacks attached to non-void functions, an optional reference parameter matching the return type can be declared, enabling the callback to inspect or override the value returned to the caller.

3.2.2.2 Callback Reusability

Advisor: Non so se mettere questa parte su Callback Reusability

In some cases, sharing a pre/post callback across multiple distinct target functions could be useful for saving time and avoiding code duplication. Because HookDesigner binds callback arguments by matching parameter names against the target function's parameters (delegating type checking and reference binding to the C++ compiler), developers can achieve callback reusability through parameter name alignment.

As illustrated in code 3.1, if two target routines (`UpdateSensorA` and `UpdateSensorB`) operate on a sensor reading under different parameter names, the developer unifies the parameter identifier to `val` in the `signatures` block. A single callback (`ChangeSensorValue`) can then be defined to inspect or mutate `val` across both functions.

Code 3.1: C++ user logic file illustrating callback reusability.

```cpp
// @signatures-start
int UpdateSensorA(int sensorId, double val);
int UpdateSensorB(double val, int flags);
// @signatures-end


// @ChangeSensorValue-start
void ChangeSensorValue(double& val) {
  if (val > 100.0) {
    val = 100.0;
  }
}
```

Correspondingly, code 3.2 demonstrates how the YAML policy attaches the unified `ChangeSensorValue` callback as a pre-hook to both hooks without duplicating logic:

Code 3.2: YAML policy snippet attaching a shared callback across multiple hooks.

```yaml
- function_name: UpdateSensorA
  behaviors: { security_policy: { pre_hook: ChangeSensorValue } }
- function_name: UpdateSensorB
  behaviors: { security_policy: { pre_hook: ChangeSensorValue } }
```

To summarize the integration between declarative policy configuration and callback definitions, code 3.3 presents a complete YAML policy targeting multiple engines, combining export lookup, RVA offset resolution, and injected user defined callbacks.

Code 3.3: Example of core HookDesigner YAML policy

```yaml
configure:
  paradigm: inline_hook
  engines: [detours, frida]
  process: scada_engine.exe
  logic_file: "policy_logic.cpp"

hooks:
  - function_name: QueuePut
    module: module.dll
    address_resolution: { strategy: export }
    behaviors:
      security_policy:
```

```
      pre_hook: QueuePut_pre
      post_hook: QueuePut_post

- function_name: UpdateSensor
  module: scada_engine.exe
  address_resolution: { strategy: offset, rva_offset: "0x1A30" }
  behaviors:
    security_policy:
      pre_hook: ChangeSensorValue

- function_name: RaiseException
  module: kernel32.dll
  is_noreturn: true
  address_resolution: { strategy: export }
```

The corresponding companion `logic_file` (`policy_logic.cpp`) is shown in code 3.4. It illustrates the signature declarations block, tagged callback implementations with parameter mutation, and return value overriding.

Code 3.4: C++ logic file (`policy_logic.cpp`) illustrating signature declarations, tag annotations, argument mutation by reference, and return value overrides.

```cpp
#include <windows.h>
#include <iostream>

// @signatures-start
int WINAPI QueuePut(void* pQueue, void* pEntry);
int UpdateSensor(int sensorId, double val);
void WINAPI RaiseException(DWORD dwExceptionCode, DWORD dwExceptionFlags,
DWORD nNumberOfArguments, const ULONG_PTR* lpArguments);
// @signatures-end

// @QueuePut_pre-start
void QueuePut_pre(void*& pQueue, void*& pEntry) {
  if (pEntry == nullptr) {
    std::cout << "Blocked null entry in QueuePut\n";
  }
}

// @QueuePut_post-start
void QueuePut_post(void* pQueue, void* pEntry, int& __hook_result) {
  if (__hook_result != 0) {
    std::cout << "[POST-LOGIC] Overriding failed QueuePut return code\n";
    __hook_result = 0;
  }
}

// @ChangeSensorValue-start
void ChangeSensorValue(double& val) {
```

```
    if (val > 100.0) {
      val = 100.0;
    }
  }
```

After signature extraction and semantic validation, for each declared hook, the transpiler matches its `function_name` against the extracted signatures, extending the hook's in-memory representation with its C++ return type, parameter list, and calling convention. This merged data structure is subsequently passed to the Jinja2 template rendering engine in Stage 3 (Section 3.3).

3.3 C++ Code Generation and DLL Compilation

Once Stage 2 has validated the declarative policy and extracted the data within the logic file, Stage 3 and Stage 4 translate the in-memory policy representation into compilable C++ source code and build the final instrumentation DLL. To realize the foundational principle of *engine-independent transpilation* established in Section 3.1.1, this phase must overcome a specific challenge: inline hooking engines expose fundamentally different low-level interfaces as examined in Inline Hooking (Section 2.4). Hard-coding engine-specific boilerplate into the transpiler would couple the transpilation process to specific frameworks, preventing the addition of new instrumentation engines without modifying the transpiler core.

To resolve this challenge, HookDesigner organizes code generation process through two different layers:

1. An *external engine registry* (`engines.yaml`) that parameterises engine-specific template selection, cast expressions, and build arguments (Section 3.3.1).
2. A *modular template hierarchy* that factors shared instrumentation logic into engine-independent partial templates while delegating specific features to engine templates (Section 3.3.2).

3.3.1 Engine Registry

The `engines.yaml` registry is a declarative configuration file that contains all engine-specific parameters and build requirements. The transpiler loads this registry at startup via `load_engines()` and uses it to drive subsequent template rendering and compilation decisions. Crucially, the registry decouples the transpiler's core code generation logic from

individual hooking backends: supporting an additional hooking engine requires only supplying its Jinja2 template and defining its parameters in `engines.yaml`, requiring zero modifications to the transpiler logic.

3.3.1.1 Registry Schema

Engine entries are organized under a *paradigm* top-level key that groups engines by their hooking model (e.g., `inline_hook`). This classification enables HookDesigner to support heterogeneous instrumentation paradigms (such as DBI backends like DynamoRIO or Intel PIN, discussed in Section 3.1.4). Table 3.4 documents the configuration schema for engines operating under the `inline_hook` paradigm, which mandates four core parameters alongside an optional list of auxiliary system libraries.

Field	Type	Req.	Role / Description
`template`	string	Yes	Filename of the engine's root Jinja2 template located in `templates/` (e.g., `detours_template.cpp`).
`cast_format`	format string	Yes	Python format string for function-pointer casting during dynamic symbol resolution (e.g., `"(True{name}_t)"` or `"(gpointer)"`).
`include_path`	string (path)	Yes	Header include directory for the hooking engine; supports `${VAR}` environment variable expansion.
`lib_path`	string (path)	Yes	Static library (`.lib`) file path for the hooking engine; supports `${VAR}` environment variable expansion.
`extra_libs`	list of strings	No	Auxiliary Windows system libraries

Table 3.4: Schema definition for engine entries registered in `engines.yaml`.

code 3.5 lists the registry configuration supporting Microsoft Detours and Frida-Gum. The `cast_format` field illustrates how the registry handles low-level engine differences without introducing conditional branching into the template files. By encoding these conventions in `cast_format`, the address resolution handler (`handle_address_resolution`, detailed in Section 3.3.3) emits the appropriate C++ cast expression directly in the transpilation process.

Code 3.5: Registry definitions for Detours and Frida-Gum in `engines.yaml`.

```
inline_hook:
  detours:
    template: "detours_template.cpp"
```

```
    cast_format: "(True{name}_t)"
    include_path: "${DETOURS_INC}"
    lib_path: "${DETOURS_LIB}"
    extra_libs:
      - "User32.lib"
      - "Advapi32.lib"

  frida:
    template: "frida_template.cpp"
    cast_format: "(gpointer)"
    include_path: "${FRIDA_INC}"
    lib_path: "${FRIDA_LIB}"
    extra_libs:
      - "User32.lib"
      - "Advapi32.lib"
      - "Ole32.lib"
      - "Shell32.lib"
```

3.3.1.2 Registry Loading and Multi-Engine Validation

The transpiler uses `load_engines()` to parse `engines.yaml`, validating each entry against paradigm schemas: `inline_hook` mandates the four required fields in Table 3.4. During policy validation (`validate_policy`), the transpiler checks every declared engine (`configure.engines`) against the registry using `get_engine_config()` helper. If a policy specifies multiple engines (e.g., `engines: ["detours", "frida"]`), the transpiler checks that all selected engines belong to the identical paradigm.

This check prevents incoherent code generation scenarios, such as simultaneously targeting two different paradigms from a single DSL policy.

3.3.2 Jinja2 template architecture

HookDesigner generates production-ready C++ instrumentation files through a modular Jinja2 [Pal24] template architecture. A naive approach to multi-target code generation would maintain monolithic, duplicated template files for each supported engine. However, with the framework we have selected, the majority of generated code is engine-independent. Duplicating these code across engines would introduce maintenance issues.

3.3.2.1 Two-Layer hierarchy

To maximize code reuse while preserving engine autonomy, in HookDesigner templates are organized into a two-layer hierarchy:

1. **Engine-Specific Templates** (`detours_template.cpp`, `frida_template.cpp`): The files that define the translation skeleton. Each template defines module-level includes, global synchronisation primitives, function-pointer typedefs, and the exported `InitializeHooks` and `DeinitializeHooks` hooks lifecycle routines (Section 3.3.5).
2. **Shared Partial Templates** (`_*.cpp`): Reusable partial templates located in `templates/`. Each file is included verbatim into the engine templates using the Jinja2 `{% include %}` directive.

Table 3.5 maps each partial template to its scope in the generated C++ code, its responsibility, and the section where its code is detailed.

Partial File	Scope	Role	Reference
`_telemetry_globals.cpp`	Global scope, above functions	Declares global telemetry handles, metric family pointers, and metric label formatting utilities.	Section 3.4
`_telemetry_init.cpp`	`InitializeHooks()` body	Initializes the logging subsystem and registers declared Prometheus metric families.	Section 3.4
`_telemetry_body.cpp`	In-hook function (post trampoline invocation)	Evaluates declared logging rules and updates performance metric observations during hook execution.	Section 3.4
`_user_logic_helpers.cpp`	Global scope, after headers	Provides runtime safety primitives: thread reentrancy guards, active thread tracking, and safe memory access helpers.	Section 3.5
`_hook_body.cpp`	Inside each `Hooked<F>` function	Implements the required instrumentation payload for function F.	Section 3.3.4
`_dllmain.cpp`	end of generated file	Minimal DLL entry point	Section 3.6

Table 3.5: Template files and correspondent responsibility.

Unlike the standard DLL pattern presented in Section 2.2.3 (where hook attachment and detachment are performed within `DllMain`), HookDesigner intentionally moves payload activation and ejection into standalone exported functions. This separation is an architectural prerequisite to prevent Windows loader-lock deadlocks when installing complex hooks, as analyzed in Section 3.6.1.3.

Crucially, the bodies of `InitializeHooks` and `DeinitializeHooks` cannot be shared across engines: as established in Section 2.4.5, guaranteeing thread safety during hook attachment and detachment requires engine-specific transactional models (such as Microsoft Detours' thread-enlisted prologue rewriting vs. Frida-Gum's dynamic interceptor runtime).

3.3.2.2 Render Context

Both engine-specific templates receive from the transpiler a *render dictionary* consisting of three entries:

- `hooks`: the list of hooks extracted in Stage 2, containing the hook policy data merged with the data derived from logic file (Section 3.2).
- `logic_file_rel`: The normalized relative filesystem path from the output C++ file to the companion user logic file (Section 3.2.2), ensuring that the generated `#include` directive resolves correctly across arbitrary build directories.
- `telemetry`: The telemetry configuration dictionary (or `None`), for managing the conditional rendering of logging and metrics C++ code (Section 3.4.1).

3.3.3 Target Address Resolution

Before template rendering begins, the transpiler resolves the virtual address lookup expression for every declared hook and stores it in `hook["address_resolution_cpp"]`, this will be used by the hooks activation routine (`InitializeHooks()`, Section 3.3.5.1). To support both public exported APIs and proprietary functions, HookDesigner provides two resolution strategies:

- **Export Resolution:** For functions exposed in a module's Export Address Table (EAT), the transpiler generatess dynamic lookup via `GetProcAddress`, supporting custom or compiler-mangled symbol names where required.
- **Offset Resolution:** In proprietary software where internal subroutines are unexported, the transpiler generates RVA pointer arithmetic relative to the module base (`GetModuleHandleA`), preserving address validity under ASLR.

To maintain engine independence, the resulting pointer expression is formatted according to the active engine's `cast_format` (e.g., typed function pointers for Microsoft Detours vs. `gpointer` for Frida-Gum, as defined in Table 3.4), ensuring that templates remain decoupled from engine-specific type casting conventions.

3.3.4 Hook Function Generation

For every function declared in the policy, HookDesigner generatess a replacement function named `Hooked<FunctionName>`. As established in Section 2.1, to preserve stack alignment

and register integrity across interception boundaries, the hooked function must match the target function's ABI specifications exactly. While 64-bit Windows environments unify calling conventions into a single paradigm, preserving calling convention qualifiers (e.g., `__stdcall`, `__cdecl`) remains critical for multi-architecture portability and 32-bit targets.

The transpiler achieves strict fidelity by rendering the function signature directly from the metadata extracted from the logic file in Stage 2 (Section 3.2.2.1), as illustrated in the template definition in code 3.6:

Code 3.6: Hook function signature rendering template.

```
{{ hook.return_type }} {% if hook.calling_convention %}{{ hook.calling_convention }}
  {% endif %}Hooked{{ hook.function_name }}({%- for param in hook.parameters -%}{{
  param.type }} {{ param.name }}{%- if not loop.last -%}, {%- endif -%}{%- endfor
  -%})
```

While the classic inline hook function structure (Section 2.4) follows a basic three-step pattern (*pre-logic* → *trampoline dispatch* → *post-logic*), deploying such a function into a live, multi-threaded process could be dangerous: an unhandled exception inside an injected callback crashes the host application, recursive API calls cause stack overflow, in-flight threads crash during DLL unloading.

For this reason in HookDesigner, the body of each hooked function (`_hook_body.cpp`) extend this basic pattern with automated telemetry collection (Section 3.4), and runtime safety defenses (Section 3.5). The complete, production in-hook execution structure is showed in detail in Section 3.5.4.

3.3.5 Hook Lifecycle Function Generation

In addition to individual hook functions (Section 3.3.4), the generated code must provide entrypoints responsible for orchestrating the overall lifecycle of the instrumentation DLL. HookDesigner generate two exported functions with standard C linkage: `Initialize-Hooks()`, which resolves targets and installs the hooks upon injection, and `Deinitialize-Hooks()`, which detaches them prior to unloading.

3.3.5.1 Transactional Attachment and Detachment

Once the target functions are located, the template performs batch hook registration using the engine's transactional API, building on the transactional patching principles established in Section 2.4.5. Transactional execution combined with the engine-specific rollback mechanisms (code 3.14) ensures atomicity: either all valid hooks are committed simultaneously, or the entire transaction fails without leaving the process in a partially patched,

unstable state.

Symmetrically, during payload deinitialization, an atomic detachment transaction reverts the patches and restores the original instructions.

Depending on the active hooking engine, the template emits the corresponding transactional loops across the declared hooks, as illustrated in code 3.7 for Microsoft Detours and code 3.8 for Frida-Gum.

Code 3.7: Simplified Jinja2 template logic for Microsoft Detours hook lifecycle functions.

```
extern "C" __declspec(dllexport) DWORD WINAPI InitializeHooks(LPVOID lpParam) {
    try {
        // Locate Functions
        {% for hook in hooks %}
        True{{ hook.function_name }} = {{ hook.address_resolution_cpp }};
        {% endfor %}

        // Transactional Attachment...


        {% for hook in hooks %}
        if (True{{ hook.function_name }}) {
            DetourAttach(&(PVOID&)True{{ hook.function_name }}, (PVOID)Hooked{{ hook.
    function_name }});
        }
        {% endfor %}

        //Transaction commit and thread cleanup...


    } catch (const std::exception& ex) {
        //diagnostic and rollback mechanism...
    }
}

extern "C" __declspec(dllexport) DWORD WINAPI DeinitializeHooks(LPVOID lpParam) {
    try {
            // Transactional Detachment...

            {% for hook in hooks %}
            if (True{{ hook.function_name }}) {
                DetourDetach(&(PVOID&)True{{ hook.function_name }}, (PVOID)Hooked{{
    hook.function_name }});
            }
            {% endfor %}

            //Transaction commit and thread cleanup...
```

```
    } catch (const std::exception& ex) {
        //diagnostic and rollback mechanism...
    }
}
```

Code 3.8: Simplified Jinja2 template logic for Frida-Gum hook lifecycle functions.

```
extern "C" __declspec(dllexport) DWORD WINAPI InitializeHooks(LPVOID lpParam) {
    try {
        gum_init_embedded();
        interceptor = gum_interceptor_obtain();

        // Locate Functions
        {% for hook in hooks %}
        target_{{ hook.function_name }} = {{ hook.address_resolution_cpp }};
        {% endfor %}

        // Transactional Replacement...
        gum_interceptor_begin_transaction(interceptor);
        {% for hook in hooks %}
        if (target_{{ hook.function_name }}) {
        gum_interceptor_replace(interceptor,
                                target_{{ hook.function_name }},
                                (gpointer)Hooked{{ hook.function_name }},
                                nullptr,
                                (gpointer*)&True{{ hook.function_name }});
        }
        {% endfor %}
        gum_interceptor_end_transaction(interceptor);

    } catch (const std::exception& ex) {
        //diagnostic and rollback mechanism...
    }
}

extern "C" __declspec(dllexport) DWORD WINAPI DeinitializeHooks(LPVOID lpParam) {
    try {
      // Transactional Detachment...
      gum_interceptor_begin_transaction(interceptor);
      {% for hook in hooks %}
      if (target_{{ hook.function_name }} != nullptr) {
        gum_interceptor_revert(interceptor, target_{{ hook.function_name }});
      }
      {% endfor %}
      gum_interceptor_end_transaction(interceptor);

      g_object_unref(interceptor);
```

```
      gum_deinit_embedded();

    } catch (const std::exception& ex) {
        //logs and rollback mechanism
    }
}
```

While codes 3.7 and 3.8 illustrate the simplified Jinja2 template loops used to generate batch attachment and detachment, deploying these routines into a live, multi-threaded process requires additional safeguards to coordinate thread synchronization, handle telemetry subsystems, and prevent loader-lock deadlocks. The complete definition of `InitializeHooks()` and `DeinitializeHooks()` is detailed in Section 3.6.

3.3.6 DLL Compilation

The generated C++ is compiled into a 64-bit (or 32-bit) Windows DLL by `compile_dll()`. The transpiler locates the MSVC compiler by invoking Microsoft's `vcvars64.bat` environment script through a spawned subprocess, eliminating the requirement for manual configuration. To assemble the complete build command, `compile_dll()` combines the compiler flags in Table 3.6 with the engine-specific settings retrieved from `engines.yaml` (Section 3.3.1).

Table 3.6 documents the specific compiler flags applied to `cl.exe` and their justifications.

Flag	Category	Behavior
`/LD`	Linker Mode	Instructs MSVC to produce a Dynamic-Link Library (DLL)
`/EHa`	Exception Model	Enables asynchronous structured exception handling (SEH) translation. (Section 3.5.1).
`/O2`	Optimization	Minimizes Interception Overhead on APIs called at high frequencies
`/utf-8`	Source Encoding	Enforces UTF-8 character encoding across source files, ensuring cross-platform consistency.
`/std:c++17`	Language Standard	Enables C++17 language constructs
`rt-link`	CRT Linkage	{/MT,/MD}, default /MT. Statically/Dynamically links the C Runtime Library (`libcmt.lib`).

Table 3.6: MSVC compiler flags utilized in `compile_dll()` and their behavior

3.3.6.0.1 DLL portability A key design constraint for HookDesigner is operational reliability in restricted industrial supervisory control and data acquisition (SCADA) settings.

If the instrumentation DLL were dynamically linked against the universal CRT (`/MD`), loading the payload onto a target machine lacking the exact corresponding redistributable build would trigger a loader failure, silently failing injection. By enforcing static CRT linkage (`/MT`) and statically linking the hooking engine (Detours or Frida-Gum) alongside the telemetry libraries (`spdlog` and `prometheus-cpp`), the transpiler outputs a completely self-contained binary. The resulting DLL carries zero dependencies beyond core Windows system libraries, guaranteeing reliable injection into restricted industrial environments.

3.4 HookDesigner Telemetry

In industrial SCADA environment, critical functions frequently execute thousands of times per second across concurrent worker threads. When instrumenting such processes, where performances are essential, telemetry cannot come at the expense of host responsiveness. A naive instrumentation strategy that performs heavy tasks (such as executing blocking disk writes, or querying network endpoints directly within the hook function) introduces latency spikes and thread contention, destabilizing the monitored system.

To overcome these constraints and fulfill the *High-Performance Telemetry* design principle introduced in Section 3.1.1, HookDesigner implements the following solutions:

1. **Asynchronous Logging (Section 3.4.4):** A non-blocking logging mechanism powered by `spdlog`, utilizing a pre-allocated bounded ring buffer that decouples hot-path hook execution from background I/O draining.
2. **Real-Time Performance Metrics (Section 3.4.5):** A native Prometheus metrics subsystem based on `prometheus-cpp`, exposed through an embedded HTTP server.

Before generating code, the framework processes telemetry policy rules during transpilation: Section 3.4.1 defines the declarative YAML schema and static validation constraints.

3.4.1 Telemetry Policy Specification

HookDesigner defines telemetry behaviors declaratively within the policy document introduced in Section 3.2.1. Telemetry configuration is split into two distinct components: process-wide parameters declared in the top-level `configure.telemetry` block, and hook-specific behavioral policies declared under each hook entry's `behaviors` mapping.

3.4.1.1 Global Telemetry Configuration

The top-level `configure.telemetry` mapping establishes the general settings for the logging framework and the metric server endpoint. Table 3.7 outlines the schema definitions for this block.

Field	Type	Req.	Default	Description
`logger`	string	No	`null`	Logging engine identifier (currently supports `"spdlog"`).
`logger_config`	object	No	`{}`	if `logger` not `null`, log configuration
`metrics`	string	No	`null`	Metrics engine identifier (currently supports `"prometheus"`).
`metrics_config`	object	No	`{}`	if `metrics` not `null`, HTTP exposer configuration.
Logger Settings (`logger_config`)				
`type`	string	No	`"file"`	Log destination: `"file"` (local disk) or `"udp"` (remote Syslog).
`host`	string	No	`"127.0.0.1"`	Target host IPv4 address only if `type: "udp"`.
`port`	integer	No	`1514`	Target UDP listening port only if `type: "udp"`.
`file`	string	No	`"hook_log.txt"`	Destination file path only if `type: "file"`.
`level`	string	No	`"info"`	Minimum severity filter (`trace`, `debug`, `info`, `warn`, `error`, `critical`).
Metrics Exposer Endpoint (`metrics_config`)				
`endpoint`	string	No	`"127.0.0.1:8080"`	Address and port bound by the embedded Prometheus HTTP server.

Table 3.7: Schema definition for the top-level `configure.telemetry` block.

3.4.1.2 Hook-Level Telemetry Behaviors

At the granularity of individual hooks, the `behaviors.performance_metrics/log_rules` defines logging and performance metrics rules to inject in the `hook.target_function`. Hook-level telemetry is configured across two sub-mappings:

- `log_rules`: A list of conditional rule objects specifying log severity, triggering conditions, and emitted message formats.
- `performance_metrics`: flag for execution latency tracking and Prometheus canonical metrics configuration.

Within the metrics configuration, an optional `label` key-value mapping can be declared.

In Prometheus, labels (Section 3.4.5.2) allow a single metric family to partition its observations dynamically according to runtime execution context (e.g., classifying a function invocation counter by return status or error code).

Table 3.8 documents the schema for these behavioral sub-policies.

Field	Type	Req.	Default	Description
Performance Metrics Policy (`behaviors.performance_metrics`)				
`execution_time`	bool	No	`false`	Enables `target_function __hook_latency` measurement
`counters`	list	No	`[]`	Counter metric definitions
`gauges`	list	No	`[]`	Gauge metric definitions
`histograms`	list	No	`[]`	Histogram metric definitions
Metric Instrument Schema (Counters, Gauges, Histograms)				
`name`	string	Yes	—	Unique metric identifier within the hook function.
`description`	string	No	`""`	Human-readable help string exported to the Prometheus registry.
`condition`	string	No	`"true"`	Boolean expression indicating when the observation occurs.
`value`	string	Gauges/Hists	—	C++ numerical expression to record (e.g., `__hook_latency`, parameter).
`buckets`	list	No (Hists)	*default*	Custom strictly increasing positive bucket upper bounds only for histograms.
`label`	dict	No	`{}`	Dynamic key-value label mapping for dimensional partitioning.
Logging Rule Schema (`behaviors.log_rules`)				
`name`	string	No	`"rule_N"`	Unique rule name identifier within the hook
`level`	string	No	`"info"`	Log severity level (`trace`, `debug`, `info`, `warn`, `error`, `critical`).
`condition`	string	No	`"true"`	Boolean expression evaluated at runtime to trigger logging.
`arguments`	list	No	`[]`	Parameter identifiers captured for auto-generated format strings.
`format`	string	No	`null`	Custom message format string supporting positional format specifierws.

Table 3.8: Schema definition for hook-level logs and metrics

3.4.1.3 Logging Rule Declaration Modalities

In DBI, logging formats can vary widely across different inspection environments. During exploratory reverse engineering, an analyst often requires rapid, call-graph tracing log to understand execution control flow. Conversely, during an incident response, logs should be able to capture precise parameter states and outcome codes formatted to match external ingestion tools.

To reconcile these different requirements, HookDesigner supports different declaration modalities within `behaviors.log_rules`, illustrated in code 3.9:

1. **Default / Empty Rule (`- {}`):** Injects a function invocation log using default metadata (`name: "rule_1"`, `level: "info"`, `condition: "true"`), only for checking if the target_function is called.
2. **Default Rule with Arguments:** The policy author can specify target function parameters to include in the log; the transpiler validates argument identifiers against the target signature and generates a structured, pipe-delimited key-value format string automatically.
3. **Custom Format String:** Parses tokens matching `{placeholder}` (supporting format specifiers like `:x` or `:.2f`) and emits a pre-compiled positional `fmtlib` template representing the custom log.
4. **Multi-Rule:** Emits independent sequential C++ `if` blocks for generating logs evaluated against return codes or runtime arguments.
5. **No Logging (`log_rules: []`):** Completely eliminates all logging code generation

Code 3.9: Demonstration of the logging rule declaration modalities

```
hooks:
  # Modality 1: Empty Rule
  - function_name: CreateFileW
    module: "kernel32.dll"
    behaviors:
      log_rules:
        - {}  # Emits: "[HOOK CreateFileW] [rule_1] called"

  # Modality 2: Default Rule with Arguments
  - function_name: UpdatePacket
    module: "user32.dll"
    behaviors:
      log_rules:
        - name: "dpi_trace"
          level: "debug"
```

```
        arguments:
          - input_hWnd
          - input_dwFlags
        # Emits: "[HOOK SetProcessDPIAware] [dpi_trace] | input_hWnd={hWnd} |
  input_dwFlags={flags}"

# Modality 3: Custom format string with format specifiers (:x)
- function_name: RegOpenKeyExA
  module: "advapi32.dll"
  behaviors:
    log_rules:
      - name: "reg_failure"
        condition: "__hook_result != 0"
        level: "warn"
        format: "[WARN] RegOpenKeyExA FAILED | key={input_lpSubKey} access=0x{
  input_samDesired:x} err={__hook_result}"
        # Emits: "[WARN] RegOpenKeyExA FAILED | key=12 access=0x20019 err=2"

# Modality 4: Multi-rule
- function_name: DispatchPacket
  module: "scada_engine.exe"
  behaviors:
    log_rules:
      - name: "on_success"
        condition: "__hook_result == 0"
        level: "debug"
        arguments:
          - input_packetId
        # Emits on success: "[HOOK DispatchPacket] [on_success] | input_packetId=13"
      - name: "on_failure"
        condition: "__hook_result != 0"
        level: "error"
        format: "[ERROR] Dispatch failed | id={input_packetId} code={__hook_result}
  latency={latency:.2f}ms"
        # Emits on failure: "[ERROR] Dispatch failed | id=101 code=-1 latency=12.45ms
  "

# Modality 5: No logging
- function_name: HighFrequencyTimerTick
  module: "scada_engine.exe"
  behaviors:
    log_rules: []  # Emits: nothing
```

code 3.10 provides a YAML policy snippet demonstrating the integration of latency tracking, Prometheus counters and histograms with dynamic labels, and multi-condition log routing.

Code 3.10: Example YAML with telemetry configuration

```yaml
configure:
  process: "scada_engine.exe"
  logic_file: "scada_logic.cpp"
  telemetry:
    logger: "spdlog"
    logger_config:
      type: "udp"
      host: {ip_addr}
      port: 1514
      level: "info"
    metrics: "prometheus"
    metrics_config:
      endpoint: "0.0.0.0:8085"

hooks:
  - function_name: ParseProprietaryPacket
    module: "scada_engine.exe"
    address_resolution:
      strategy: export
    behaviors:
      performance_metrics:
        execution_time: true
        counters:
          - name: "packets_total"
            description: "Total proprietary packets processed"
            label:
              status: "__hook_result == 1 ? success : failure"
        histograms:
          - name: "processing_latency_ms"
            description: "Distribution of packet parsing latency"
            value: "__hook_latency"
            buckets: [0.05, 0.1, 0.5, 1.0, 5.0, 10.0]
            label:
              status: "__hook_result == 1 ? success : failure"
      log_rules:
        - name: "packet_failure"
          condition: "__hook_result != 1"
          level: "warn"
          format: "[WARN] Packet parse failure | weight={input_complexityWeight} thread
    ={__hook_tid} latency={__hook_latency:.2f}ms"
```

3.4.2 Static Telemetry Expression Translation

Inside a detour function, evaluating telemetry expressions—such as conditional triggers, custom format strings, and dynamic metric values—through runtime interpretation would be inefficient. While conventional frameworks rely on embedded interpreters or runtime

Algorithm 1 Static telemetry expression translation (`translate_telemetry_expr`)

Require: telemetry expression e, target hook metadata h
Ensure: translated C++ expression e, updated capture sets and tracking flags on h

1: replace every `__hook_tid` in e with `GetCurrentThreadId()`
2: **if** `__hook_latency` occurs in e **then**
3: **if** h.`is_noreturn` **then**
4: **throw** semantic error
5: **end if**
6: mark h as requiring execution-time tracking
7: replace `__hook_latency` with `elapsed_ms_<`h.`function_name>`
8: **end if**
9: **if** `__hook_result` occurs in e **then**
10: **if** h.`return_type` is `void` **or** h.`is_noreturn` **then**
11: **throw** semantic error
12: **end if**
13: **end if**
14: **for** each token `input_<p>` or `output_<p>` in e **do**
15: **if** $p \notin h$.`parameters` **or** (token is `output_` **and** h.`is_noreturn`) **then**
16: **throw** semantic error
17: **end if**
18: register p in h.`input_vars_needed` (or h.`output_vars_needed`)
19: **end for**
20: **return** e

Abstract Syntax Tree evaluators—introducing C-structure marshaling costs, heap allocations, and garbage collection jitter per each hook invocation—HookDesigner adopts a *static expression translation* approach across Stage 2 and Stage 3. The transpiler analyzes telemetry expressions ahead of time and translates them directly into native C++ instructions. The core translation and demand-driven parameter capture registration procedure is formalized in Algorithm 1.

3.4.2.1 Telemetry Wildcards

As formalized in lines 2–9 of Algorithm 1, domain-specific wildcards in telemetry expressions are resolved directly into concrete C++ constructs:

- `__hook_tid` (line 2) rewrites to a Win32 `GetCurrentThreadId()` call.
- `__hook_latency` (lines 3–6) automatically forces execution time tracking on target hook h and rewrites the symbol to `elapsed_ms_<FunctionName>`, statically rejecting

non-returning routines.

- `__hook_result` (lines 7–9) maps to the target routine's return value, rejecting functions that are `void` or non-returning.

3.4.2.2 Parameter Capturing

A requirement in inline hooking is capturing function parameter values accurately across a detour function execution (Section 3.3.4). Target functions or injected callbacks may mutate parameter buffers or pointer values in-place during execution. Consequently, a telemetry rule evaluated *after* the original function returns may observe altered parameter rather than the values originally supplied by the caller.

To provide correct visibility, HookDesigner establishes an explicit way for targeting parameter from the policy file:

- `input_<param>`: Represents the parameter captured upon hook entry, *before* any pre-hook callbacks or original function execute.
- `output_<param>`: Represents the parameter captured *after* the original function returns and post-hook execution.

Crucially, argument capturing is *demand-driven*. By scanning telemetry expressions with regular expressions (`_VAR_RE`), Algorithm 1 (lines 10–14) identifies every referenced parameter, validates it against the formal signature of h (rejecting post-invocation captures on non-returning routines), and registers it into the corresponding capture set (`input_vars_needed` or `output_vars_needed`). If an argument is never referenced in telemetry, no capture code is emitted for it.

For every needed parameter, the transpiler invokes `analyze_type()` on its Clang type handle (`_clang_type`, preserved from the signature extraction stage in Section 3.2.2.1), to automatically generate the safe C++ code for saving the parameter's value according to its type, as detailed in Section 3.5.2:

- **C-Style Strings:** The transpiler generates safe string capture via `input_<param> = CopyString(<param>)` helper
- **Primitive Pointers:** The transpiler generates safe pointer dereference chains via `input_<param> = ReadPointer(<param>)` helper.
- **Primitive Values:** Value types are captured directly by value assignment.

3.4.2.2.1 Member Field Access A design trade-off arises when policy rules reference struct members (e.g., `{input_pQueue->queueId}` or `{input_pStruct.value}`). When member operators (`->` or `.`) are detected, the transpiler identifies the base symbol (`pQueue`) to ensure the base pointer is captured in `input_vars_needed`. However, it deliberately bypasses complex recursive CLANG struct-layout inspection on the accessed field, emitting the raw member path directly into the generated C++ code. This grants policy authors full expressiveness to inspect arbitrary proprietary structures, while deferring a safe access to the member value directly to the policy authors as detailed in Section 3.5.2.

3.4.2.3 Custom Log Format Handling

When processing a custom log format (e.g., `format: "ID={input_id} Ret={__hook_result:x}"`), naive instrumentation engines format the message by parsing placeholders dynamically at runtime, introducing non-deterministic latency. Because HookDesigner's logging engine (`spdlog`, Section 3.4.4) relies on the `fmt` library (`fmtlib`) for message formatting, the transpiler completely eliminates runtime string parsing by pre-compiling declarative templates into native `fmtlib` positional format strings during code generation.

As illustrated in Figure 3.1, the transpiler scans the format string for tokens matching `{placeholder[:specifier]}`. For each detected token:

1. The inner expression is resolved into a native C++ expression via Algorithm 1 and appended to the `fmt` argument list.
2. The entire token itself in the template string is rewritten into an empty or specifier positional token (e.g., `{}` or `{:x}`), generating a positional format string.

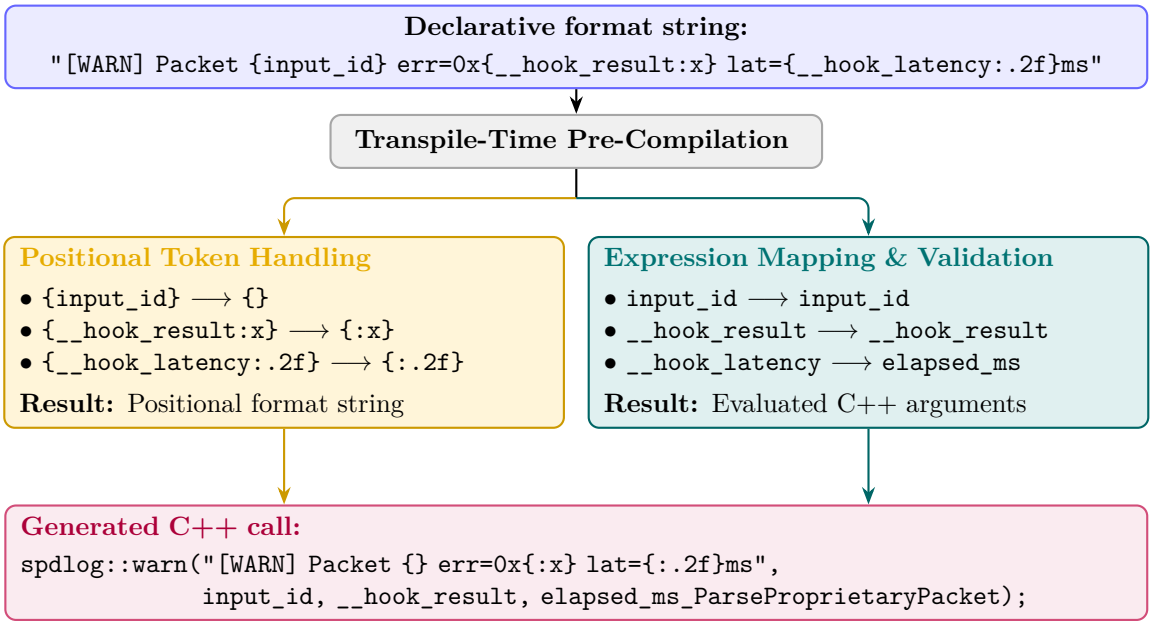


Figure 3.1: Transpile-time format string pre-compilation

At runtime, the template emits an spdlog log via `spdlog::<level>(format_str, args...)`. Format string parsing and argument type checking are executed entirely at C++ compile time by `fmtlib`, resulting in zero runtime parsing overhead within the detour function execution.

Because telemetry frequently evaluates function outcome codes (`__hook_result`), elapsed execution latency, and mutated output buffers (`output_`), HookDesigner positions the telemetry evaluation block (`_telemetry_body.cpp`) in the hook epilogue, immediately following the trampoline invocation.

For a target routine `int ProcessCommandSignal(int cmdId, double* payloadValue, char* responseBuffer)`, code 3.11 demonstrates the concrete C++ code generated by the transpiler inside `_hook_body.cpp` and `_telemetry_body.cpp` from declarative telemetry rules:

Code 3.11: Generated C++ code inside `_hook_body.cpp` and `_telemetry_body.cpp` for `ProcessCommandSignal`, demonstrating demand-driven parameter capture

```
// Entry Parameter Capturing
int input_cmdId = cmdId;
double input_payloadValue = ReadPointer(payloadValue);
std::string input_responseBuffer = CopyString(responseBuffer);

// Target function latency measurement
LARGE_INTEGER start_time_ProcessCommandSignal;
```

```
QueryPerformanceCounter(&start_time_ProcessCommandSignal);

int __hook_result = TrueProcessCommandSignal(cmdId, payloadValue, responseBuffer);

LARGE_INTEGER end_time_ProcessCommandSignal;
QueryPerformanceCounter(&end_time_ProcessCommandSignal);
double elapsed_ms_ProcessCommandSignal = (double)(end_time_ProcessCommandSignal.
    QuadPart - start_time_ProcessCommandSignal.QuadPart) * 1000.0 / (double)
    g_PerfFrequency.QuadPart;

// Telemetry code

// Exit Parameter Capturing
double output_payloadValue = ReadPointer(payloadValue);
std::string output_responseBuffer = CopyString(responseBuffer);

// Telemetry Evaluation
if (__hook_result != 200) {
    spdlog::warn("[WARN] Signal failed | id={} | payload: {:.1f} -> {:.1f} | resp: '{}'
     -> '{}' | lat={:.2f}ms",
                 input_cmdId,
                 input_payloadValue,
                 output_payloadValue,
                 input_responseBuffer,
                 output_responseBuffer,
                 elapsed_ms_ProcessCommandSignal);
}

return __hook_result;
```

3.4.3 Static Semantic Validation

During Stage 2 (`validate_policy`), HookDesigner performs semantic analysis across declared telemetry rules to catch possible errors offline before DLL compilation. Beyond standard syntactic checks (such as verifying unique rule identifiers and the mutual exclusivity of `arguments` and `format`), the transpiler enforces semantic constraints:

1. **Non-Returning Functions:** For target routines classified as non-returning (`is_noreturn: true`), execution never returns after trampoline invocation. The validator statically rejects any telemetry specification on these targets that references post-invocation properties (including `execution_time: true`, `__hook_latency`, `output_<param>`, or `__hook_result`), preventing the generation of dead code.
2. **Custom Histogram Buckets Monotonicity:** Prometheus registry initialization fails at runtime if histogram bucket upper bounds are unordered or non-positive. The

transpiler checks custom `buckets` arrays at compile time to ensure strict positive monotonicity, preventing silent runtime metric registration.

3. **Bounded Label Count:** A possible failure in metric framework is *cardinality explosion*: the unbounded allocation of metric series caused by assigning high-cardinality values to metric labels or by specifying a huge number of different label per metric. HookDesigner reduces this risk by strictly restricting metric definitions to at most a single dynamic label, redirecting high-cardinality inspection to structured log events.

3.4.4 Asynchronous Logging

Logging in high performance environment can be subjected to timing constraints. Standard synchronous logging models serialize execution by acquiring shared locks and performing blocking operations (such as disk writes or network system calls) directly on the critical execution path. In an industrial process where intercepted APIs are invoked at high frequencies across multiple threads, synchronous logging often introduces stalls in worker threads, degrading performances of the software under observation.

3.4.4.1 Ring Buffer Architecture

HookDesigner prevents this possible overhead by integrating asynchronous logging powered by `spdlog`. During hook initialization in `_telemetry_init.cpp`, the framework pre-allocates an asynchronous thread pool:

```
spdlog::init_thread_pool(65536, 1);
```

This initializes a pre-allocated circular queue capable of holding up to an arbitrary number of pending log items (in our case is 65,536), consumed by a dedicated background worker thread, as modeled in Figure 3.2.

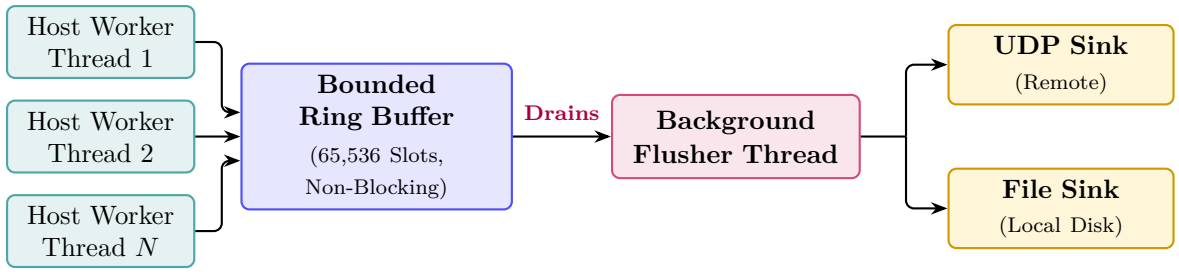


Figure 3.2: Architecture of the asynchronous structured logging subsystem

When a hook function triggers a log rule, the application thread executes only the minimal CPU instructions required to format the argument tuple and push the record into the circular queue. No disk I/O, network transmission occurs on the application thread. The dedicated background thread drains records from the queue and transmits them to the active target, guaranteeing that logging overhead on the host threads remains strictly bounded to memory speed.

3.4.4.2 Buffer Saturation Policy

Under burst load, if an application thread generates log messages faster than the background thread can write them to disk or network, a standard logging system will block the calling thread until queue space becomes available (inducing overhead). With a pre-allocated capacity of 65,536 (2^{16}) entries, the circular ring buffer is intentionally oversized, easily handling standard industrial traffic burst without dropping a single event. Saturating a queue of this size is unlikely under normal operating conditions; it represents extreme scenario—such as complete physical disk stalls or total network interface disconnection. In the rare event that saturation does occur, HookDesigner avoids this by configuring the asynchronous logger with the `overrun_oldest` policy:

```
spdlog::async_overflow_policy::overrun_oldest
```

Under queue saturation, if the 65,536-entry queue fills completely, the logger drops the oldest unwritten messages to make space for incoming events. This guarantees that application worker threads are *never* blocked by logs operations, following the minimal-overhead telemetry design principle established in Section 3.1.

3.4.4.3 Network and Storage Sinks

The logging subsystem supports two distinct log destination (sink):

- **UDP Sink (`logger_config.type: "udp"`)**: Industrial deployments frequently aggregate telemetry across distributed nodes into monitoring systems (such as Promtail, Grafana Loki). HookDesigner integrates a non-blocking UDP sink using `spdlog::sinks::udp_sink_mt`. Utilizing connectionless UDP transport provides an advantage: if the remote logging server crashes or network connectivity is down, the operating system immediately discards the packets. The hook payload never blocks attempting to re-establish TCP handshakes, isolating the host process from external network failures.
- **Local File Sink (`logger_config.type: "file"`):** HookDesigner instantiates also a asynchronous basic file sink (`spdlog::sinks::basic_file_sink_mt`).

File flushing is explicitly triggered only on error conditions (`spdlog::flush_on(spdlog::level::err)`) to minimize disk write amplification during a possible host process crash.

3.4.5 Performance Metrics Engine

While asynchronous logging provides textual event traces, monitoring of industrial software requires also quantitative analysis (e.g. request throughput, error ratios, and latency distributions). Extracting these data from log streams requires external log parsing, which could be computationally expensive.

To provide *instantaneous* quantitative observability, HookDesigner integrates Prometheus metrics exposition via `prometheus-cpp`. The instrumentation payload hosts an embedded Prometheus HTTP server that continuously exposes current metric values to external scrapers.

3.4.5.1 Registration and Metric Family Hierarchy

During hook payload initialization (`InitializeHooks`), the framework allocates a global metric registry and starts an embedded CivetWeb-based HTTP exposer on the configured socket address:

```
g_PromRegistry = std::make_shared<prometheus::Registry>();
g_PromExposer = std::make_unique<prometheus::Exposer>("telemetry.metrics_config.
    endpoint");
g_PromExposer->RegisterCollectable(g_PromRegistry);
```

For every metric declared under `behaviors.performance_metrics`, HookDesigner generates typed global metric family pointers (`g_PromFamily_<Function>_<Metric>`) bound to the central registry:

- **Counters (`BuildCounter`):** Monotonically increasing values used to track event frequencies (e.g., total packets processed, errors encountered).
- **Gauges (`BuildGauge`):** Arbitrary numerical values that can increase or decrease, representing instantaneous states (e.g., active queue depth, buffer capacity).
- **Histograms (`BuildHistogram`):** Samples observations into configurable statistical buckets alongside cumulative counts and sums

3.4.5.2 Dynamic Labels

Prometheus metrics achieve multi-dimensional data modeling through key-value labels. Intercepted function calls rarely carry uniform monitored value; an analyst must differentiate observations based on runtime execution context (e.g. distinguishing normal completions from error codes, tracking request w.r.t. specific value of a variable).

Without labels, an analyst would be forced to define separate, metric families for every potential outcome (e.g., `packets_success_total`, `packets_auth_error_total`, `packets_crc_error_total`). This practice pollutes the metric registry and requires continuous re-compilation whenever new statuses arise. By attaching dynamic labels (e.g., `packets_total{status="success"}`), a single metric family can be dynamically partitioned at runtime according to the specified key (variable).

3.4.5.2.1 Lock Contention Bottleneck While dynamic labels provide monitoring dynamic granularity, implementing them efficiently presents a performance challenge. In the standard `prometheus-cpp` library, observing a metric with dynamic labels requires invoking:

```
family->Add({ {"label_key", label_value} }).Increment();
```

Under the hood, `prometheus::Family<T>::Add()` maintains an internal structure protected by a lock. Every time a hook executes an observation, the thread must:

1. Construct a node containing the label key-value pair, triggering heap allocations.
2. Acquire the family's internal lock.
3. Perform an associative tree search across registered label sets.
4. Release the lock and return the metric reference.

In a multi-threaded application where worker threads execute thousands of operations per second, this design induces *lock contention*. Threads serialize on the shared lock, introducing latency into the host application.

3.4.5.2.2 The Lock-Free Thread-Local Cache HookDesigner overcomes this bottleneck by implementing a *Caching strategy* generated into `_telemetry_body.cpp`.

Behind this optimization is *temporal label locality*: within industrial processes, worker threads frequently execute batches of operations that share identical label values over

extended time windows (e.g., a worker thread processing consecutive packets belonging to the same connection ID).

To exploit this locality, the transpiler generates two thread-local variables for every labeled metric, accessible without acquiring any lock:

1. `thread_local uint64_t tl_last_hash`: Stores the 64-bit hash of the last observed label value on that specific thread.
2. `thread_local MetricType* tl_last_ptr`: Stores a direct pointer to the underlying Prometheus metric instance.

code 3.12 illustrates the caching logic generated by the transpiler.

Code 3.12: Zero-lock thread-local caching mechanism

```
if (g_PromFamily_ParseProprietaryPacket_packets_total) {
    if (__hook_result == 1) {
        // Evaluate label string and compute 64-bit hash
        std::string cur_val = ToLabelString("success");
        uint64_t cur_hash = std::hash<std::string>{}(cur_val);

        // Thread-local storage variables
        thread_local uint64_t tl_last_hash_packets_total = 0;
        thread_local prometheus::Counter* tl_last_ptr_packets_total = nullptr;

        // Fast-Path Check: bypass mutex if label matches cached state
        if (tl_last_ptr_packets_total == nullptr || tl_last_hash_packets_total !=
    cur_hash) {
            // SLOW PATH: Mutex acquired only on start or label transition
            tl_last_ptr_packets_total = &
    g_PromFamily_ParseProprietaryPacket_packets_total->Add({ {"status", cur_val} });
            tl_last_hash_packets_total = cur_hash;
        }

        // FAST PATH: Lock-free atomic increment
        tl_last_ptr_packets_total->Increment();
    }
}
```

3.4.5.3 Direct Binding for Label-less Metrics

For metrics defined without dynamic labels, HookDesigner applies a compile-time optimization. Because the metric dimensionality is static, resolving the metric instance dynamically at runtime is entirely redundant. During initialization (`_telemetry_init.cpp`),

the transpiler generatess code that registers the metric family and immediately adds the base instance, storing the result in a static global pointer:

```
g_PromMetric_ParseProprietaryPacket_total = &g_PromFamily_ParseProprietaryPacket_total
    ->Add({ {"function", "ParseProprietaryPacket"} });
```

Inside the hook function (`_telemetry_body.cpp`), the observation bypasses even the thread-local hash check, updating the metric directly through the global pointer:

```
if (g_PromMetric_ParseProprietaryPacket_total) {
    g_PromMetric_ParseProprietaryPacket_total->Increment();
}
```

This delivers maximum execution efficiency, reducing metric recording overhead to a single atomic instruction.

To demonstrate how these heterogeneous telemetry features operate cohesively within an intercepted function, code 3.13 illustrates the complete in-hook telemetry evaluation block generated with `_telemetry_body.cpp`. In this example we can see spdlog logging, counters, cached labeled histograms, and state gauges that HookDesigner handle with minimal overhead.

Code 3.13: Generated in-hook telemetry block (`_telemetry_body.cpp`) illustrating the co-existence of the seen telemetry features

```
  // Asynchronous Logging
  if (__hook_result != 1) {
    spdlog::warn("[WARN] Dispatch failed | id={} | code={} | lat={:.2f}ms",
    input_packetId,
    __hook_result,
    elapsed_ms_DispatchPacket);
  }

  // Metric Update (No label)
  if (g_PromMetric_DispatchPacket_invocations_failed) {
    if (__hook_result != 1) {
      g_PromMetric_DispatchPacket_invocations_failed->Increment();
    }
  }

  // Dynamic-Label Histogram
  if (g_PromFamily_DispatchPacket_latency_ms) {
    std::string cur_val = ToLabelString(__hook_result == 1 ? "success" : "failure");
    uint64_t cur_hash = std::hash<std::string>{}(cur_val);

    thread_local uint64_t tl_last_hash_latency_ms = 0;
    thread_local prometheus::Histogram* tl_last_ptr_latency_ms = nullptr;

    if (tl_last_ptr_latency_ms == nullptr || tl_last_hash_latency_ms != cur_hash) {
```

```
      tl_last_ptr_latency_ms = &g_PromFamily_DispatchPacket_latency_ms->Add(
      { {"status", cur_val} },
      g_PromBuckets_DispatchPacket_latency_ms
      );
      tl_last_hash_latency_ms = cur_hash;
    }
    tl_last_ptr_latency_ms->Observe(elapsed_ms_DispatchPacket);
  }

  // Gauge
  if (g_PromMetric_DispatchPacket_queue_depth) {
    g_PromMetric_DispatchPacket_queue_depth->Set((double)(g_active_queue_depth.load()))
    ;
  }
```

3.5 DLL Runtime Safety

As established in Section 2.2.2, during DBI, the compiled payload runs directly inside the target application's virtual address space without memory isolation boundaries. While this shared-memory model enables DBI, it introduces several runtime safety issues: a fault or unhandled exception anywhere within the injected DLL—whether during lifecycle orchestration (`InitializeHooks`, `DeinitializeHooks`) or inside hook function execution (`_hook_body.cpp`)—immediately crashes the host process. In production environments, such unexpected terminations violate the *Crash Safety* design principle established in Section 3.1.1.

To satisfy this requirement, HookDesigner incorporates a protection mechanisms in both the DLL lifecycle routines and the hook function body:

1. **Asynchronous Exception Handling:** By compiling the DLL under the MSVC `/EHa` model, both C++ exceptions and OS hardware faults (SEH) are intercepted across lifecycle routines and hook wrappers, preventing host process termination (Section 3.5.1).

2. **Memory Access Helpers:** The transpiler generates safe pointer dereference and string copying primitives that protect against reading invalid memory during parameter capture (Section 3.5.2).

3. **Thread-Local Recursion Guard:** A TLS guard prevents re-entrant hook execution loops caused by circular API invocations on the same thread (Section 3.5.3).

4. **Complete Hook Function Structure:** this structure shows how the presented

defenses are integrated within the definitive Hook Function implementation, with specializations for both standard and non-returning functions (Section 3.5.4).

3.5.1 Asynchronous Exception Handling

In standard C++ development on Windows, compilers typically use the synchronous exception model (`/EHsc`) [Mic26a]. Under `/EHsc`, C++ `try/catch` blocks are designed only to intercept exceptions thrown explicitly through the C++ `throw` statement. Under this standard configuration, a C++ `catch(...)` block does not catch OS-level faults—such as dereferencing a null pointer, reading an unmapped virtual address, or encountering a division by zero. In this way, the exception bypasses user code, triggers the default operating system exception filter, and terminates the host process immediately.

3.5.1.1 The /EHa Asynchronous Exception Model

To eliminate this behavior, HookDesigner's compilation enforces the asynchronous exception handling model [Mic26a] by passing the `/EHa` flag to the MSVC (Section 3.3.6) compiler. Under `/EHa`, the compiler generates structured unwinding tables that bridge Windows kernel SEH with C++ exception handling:

- Standard C++ `catch(...)` clauses are transformed to catch both synchronous C++ exceptions and asynchronous Win32 hardware faults.
- Proper stack unwinding and destructor execution are guaranteed even when an instruction triggers an OS fault.

3.5.1.2 Exception Boundaries

To prevent cascading failures and isolate runtime faults across the DLL, HookDesigner organizes exception boundaries across two tiers:

3.5.1.2.1 DLL Lifecycle Functions Tier: The exported lifecycle routines `InitializeHooks()` and `DeinitializeHooks()` are enclosed in `try/catch` blocks compiled under `/EHa`. While the template listings in (Section 3.3.5) focused on the primary attachment and detachment workflows, production safety requires explicit transaction rollback and resource cleanup inside these exception handlers, as shown in code 3.14.

If an unhandled exception or hardware fault occurs during target address resolution, engine transaction commits, HTTP listener initialization, or worker thread flushing, the fault is

safely intercepted, logged via `OutputDebugStringA`, and mapped to an explicit failure exit code (such as `0xE0000002` for C++ exceptions or `0xE0000003` for SEH hardware faults) returned to the injector via `CallRemoteExport`. Crucially, if a fault occurs while an engine transaction is open (`in_transaction == true`), the exception handler triggers an explicit rollback to prevent partial instrumentation:

- **Microsoft Detours:** Handled natively via `DetourTransactionAbort()`, which rolls back all staged detours without modifying memory and closes all opened worker thread handles.
- **Frida-Gum:** Because Frida-Gum lacks an atomic abort primitive and `gum_interceptor_end_transaction()` commits pending memory updates, the handler explicitly disarms staged hooks via `gum_interceptor_revert()` before closing the transaction with `gum_interceptor_end_transaction()`.

Finally, partially constructed telemetry structures and TLS allocations are destroyed (Section 3.5.3). This guarantees that an in-flight failure leaves zero dangling hooks, thread suspensions, or corrupted memory pages, allowing the injector to safely execute rollback or ejection fallback handling without destabilizing the host application (Section 3.6).

Code 3.14: Atomic transaction rollback and resource containment in lifecycle exception handlers.

```
// catch blocks inside InitializeHooks and DeinitializeHooks
catch (const std::exception& ex) {
    // Engine Transaction Rollback
    // Detours:
    if (in_transaction) {
        DetourTransactionAbort();
    }
    for (HANDLE hThread : threads) {
        CloseHandle(hThread);
    }

    // Frida-Gum (explicit revert loop before concluding transaction):
    if (in_transaction && interceptor) {
        for (auto target : targets) {
            if (target) gum_interceptor_revert(interceptor, target);
        }
        gum_interceptor_end_transaction(interceptor);
        in_transaction = false;
    }
    if (interceptor) {
        g_object_unref(interceptor);
        interceptor = nullptr;
        gum_deinit_embedded();
```

```
    }

    // Teardown initialized telemetry & TLS slots
    g_PromExposer.reset();
    g_PromRegistry.reset();
    spdlog::shutdown();
    if (g_TlsIndex != TLS_OUT_OF_INDEXES) {
        TlsFree(g_TlsIndex);
        g_TlsIndex = TLS_OUT_OF_INDEXES;
    }

    std::string msg = fmt::format("[CRITICAL] Exception in lifecycle routine: {}\n", ex
    .what());
    OutputDebugStringA(msg.c_str());
    return 0xE0000002; // Custom failure code for C++ exception returned to injector
}
catch (...) {
    // Engine rollback and resource deallocation identical to above...
    OutputDebugStringA("[CRITICAL] SEH/Hardware fault in lifecycle routine\n");
    return 0xE0000003; // Custom failure code for SEH/Hardware fault returned to
    injector
}
```

3.5.1.2.2 Hook Function Tier: Inside the body of each hooked function (`_hook_body.cpp`), execution is segmented into decoupled, independent exception boundaries:

1. **Parameter Capture Boundaries:** Entry and exit argument captures are wrapped in dedicated exception boundaries. If an SEH hardware fault occurs, the error is logged with the corresponding parameter name, and execution safely continues without aborting user-defined callbacks.

2. **User Callback Boundaries:** User-defined callbacks (`pre_hook` and `post_hook`) execute inside separate, independent exception blocks. A fault within user code is contained, guaranteeing that the target function is still invoked via its trampoline (`TrueTargetFunction`), that the telemetry code is executed and that the return value is safely passed back to the caller.

3. **Telemetry Evaluation Boundary:** Telemetry evaluation is placed inside its own exception boundary, if any required parameter failed to capture, telemetry evaluation is automatically skipped. This prevents format strings and metric observations from computing over uninitialized or corrupted memory. Furthermore, any exception encountered during telemetry emission cannot destabilize the host application or corrupt the function return code.

code 3.15 illustrates this decoupled pattern:

Code 3.15: Decoupled exception boundaries in `_hook_body.cpp`

```
// Entry Parameter Capture Boundary
bool telemetry_inputs_ready = false;
const char* capturing_input_param = nullptr;
try {
    capturing_input_param = "payloadValue";
    input_payload = ReadPointer(payloadValue);
    telemetry_inputs_ready = true;
} catch (const std::exception& ex) {
    OutputDebugStringA(fmt::format("[HOOK CAPTURE] Exception in {}: {}\n",
    capturing_input_param, ex.what()).c_str());
} catch (...) {
    OutputDebugStringA(fmt::format("[HOOK CAPTURE] SEH fault in {}\n",
    capturing_input_param).c_str());
}

// Pre-Hook User Logic Boundary
try {
    UserPreCallback(payloadValue);
} catch (const std::exception& ex) {
    OutputDebugStringA("[HOOK EXCEPTION] Exception in pre-hook\n");
} catch (...) {
    OutputDebugStringA("[HOOK EXCEPTION] Hardware/SEH fault in pre-hook\n");
}

int __hook_result = TrueTargetFunction(payloadValue);

//Post-Hook User Logic Boundary
try {
    UserPostHook(__hook_result);
} catch (const std::exception& ex) {
    OutputDebugStringA("[HOOK EXCEPTION] Exception in post-hook\n");
} catch (...) {
    OutputDebugStringA("[HOOK EXCEPTION] Hardware/SEH fault in post-hook\n");
}

//Exit Parameter Capture Boundary
bool telemetry_outputs_ready = false;
const char* capturing_output_param = nullptr;
try {
    capturing_output_param = "payloadValue";
    output_payload = ReadPointer(payloadValue);
    telemetry_outputs_ready = true;
} catch (const std::exception& ex) {
    OutputDebugStringA(fmt::format("[HOOK CAPTURE] Exception in {}\n",
    capturing_output_param).c_str());
} catch (...) {
```

```
    OutputDebugStringA(fmt::format("[HOOK CAPTURE] SEH fault in {}\n",
    capturing_output_param).c_str());
}

// Telemetry Evaluation Boundary
if (telemetry_inputs_ready && telemetry_outputs_ready) {
    try {
        EvaluateTelemetry();
    } catch (const std::exception& ex) {
        OutputDebugStringA("[TELEMETRY EXCEPTION] Exception in telemetry\n");
    } catch (...) {
        OutputDebugStringA("[TELEMETRY EXCEPTION] Unknown exception in telemetry\n");
    }
}

return __hook_result;
```

3.5.2 Transpiler Type Analysis and Memory Access Helpers

As already explained in Section 3.4.2.2, DBI authors often need to inspect function parameters on function entry and exit. However, manually writing C++ code to capture parameters' value is error-prone: forgetting a null check, dereferencing an invalid pointer, or traversing an unbounded string can easily cause memory access violations. To help the work of policy authors, HookDesigner *automates* access to primitive variables and strings in a safe way. By analyzing parameter types at transpile time, the framework automatically generates the appropriate access solution, preventing the author from writing repetitive and error-prone C++ boilerplate.

3.5.2.1 Type Analysis

As introduced in Section 3.4.2.2, every parameter required for telemetry must be safely captured on function entry or exit. To automate this without requiring manual boilerplate from the user, the transpiler performs static type analysis on each parameter's Clang type representation (preserved from the Stage 1 in Section 3.2.2.1).

The analysis classifies each parameter through three conceptual phases:

- **Canonical Alias Resolution:** In C and Win32 development, types are frequently masked behind typedefs (such as `DWORD`, `LPSTR`, or `size_t`). The transpiler traverses `get_canonical()` to resolve these aliases to their underlying primitive representation.

- **Pointer Indirection Resolution:** By recursively unwinding pointee types, the transpiler calculates the pointer depth d. Types with $d = 1$ pointing to character types (`char`, `wchar_t`) are classified as string candidates, whereas non-pointer primitives have $d = 0$.
- **Safe Access Expression:** For string pointers, the transpiler generates safe copy calls (`CopyString`). For dereferenceable primitive pointers (excluding opaque handles such as `void*` and complex structures), it nests d dereference helpers (`ReadPointer`). Opaque handles and direct values are passed unchanged.

This automated classification enables the transpiler to emit uniform assignments of the form `input_<p> = safe_read_expr` without requiring template branching in the hook body. Table 3.9 formalizes the complete type mapping matrix.

Clang Type	**Depth (d)**	**Category**	**Generated Read Expression**	**Storage Type**
`int`, `double`, `DWORD`	0	Primitive Value	`param`	`T` (original type)
`const char*`, `char*`	1	C-String	`CopyString(param)`	`std::string`
`const wchar_t*`, `wchar_t*`	1	Wide C-String	`CopyString(param)`	`std::string` (UTF-8)
`int*`, `double*`	1	Primitive Pointer	`ReadPointer(param)`	Underlying type (`int`)
`int**`	2	Nested Pointer	`ReadPointer( ReadPointer(param))`	Underlying type (`int`)
`void*`, `HANDLE`	1	Opaque Handle	`param`	Raw pointer type

Table 3.9: Mapping clang type and generated expression

3.5.2.2 Memory-Safety Helper Functions

Most of the generated expressions leverage helper functions defined in `_user_logic_helpers.cpp`:

- **Pointer Dereferencing (`ReadPointer<T>`):** When reading data through a pointer,

standard C++ dereferencing (`*ptr`) crashes the process if the pointer is null or unmapped. HookDesigner defines a templated helper:

```
template <typename T>
static T ReadPointer(const T* ptr, T fallback = T()) {
    if (!ptr) {
        return fallback;
    }
    return *ptr;
}
```

If `ptr` is null, `ReadPointer` returns the default-constructed fallback value without attempting a dereference. If `ptr` contains a non-null but unmapped address (e.g., a dangling pointer pointing to deallocated memory), the resulting read violation is trapped by the outer `/EHa` exception boundary.

- **Bounded String Copying (`CopyString`):** Passing raw string pointers (`char*` or `wchar_t*`) directly to logging formatters is dangerous: if the target buffer lacks a null terminator or points to invalid memory, formatting routines read past the buffer boundary until they fault. HookDesigner generates explicit calls to `CopyString`:

```
static std::string CopyString(const char* source, size_t maxChars = 128) {
    if (!source) return "null";
    size_t len = strnlen(source, maxChars);
    return std::string(source, len);
}
```

`CopyString` checks for null pointers, returning the literal `"null"`. For valid pointers, it uses the bounded length function `strnlen(source, maxChars)`, reading at most 128 characters. For wide strings (`wchar_t*`), an overloaded implementation uses `wcsnlen` and converts the result to UTF-8 using `WideCharToMultiByte(CP_UTF8)`, ensuring consistent log formatting.

3.5.2.3 Member Access Contract

As introduced in Section 3.4.2.2, when a policy rule references an internal struct member using the arrow or dot operator (e.g., `{input_pQueue->queueId}`), the transpiler detects the operator and *bypasses automated Clang type analysis* on the accessed field. Instead, it emits the member access expression defined in the YAML policy directly into the generated C++ code.

This design establishes an explicit contract:

- **Author Flexibility:** The policy author can inspect nested member field, HookDesigner will not refuse the policy.
- **Author Responsibility:** The author is responsible for ensuring pointer or string member safety from the YAML policy, exploiting the helpers provided in `logic_helpers.cpp` (e.g. by wrapping the expression in `ReadPointer(input_pQueue->queueId)` if the field `queueId` is a pointer).
- **Exception Boundary Safety:** However, any possible error of the policy author is intercepted by the outer `/EHa` SEH containment boundary, preventing any possible host crash.

3.5.2.4 Array-to-Pointer Decay

A fundamental characteristic of the C and C++ type systems is *array-to-pointer decay*: function parameters declared as arrays decay into raw pointer types within the Clang type analysis without preserving runtime size metadata. Consequently, HookDesigner's automated analysis (Table 3.9) cannot deduce the allocated extent of an arbitrary pointer argument at transpile time without external semantic annotations.

To uphold strict memory safety and avoid out-of-bounds reads or buffer overread vulnerabilities, HookDesigner deliberately enforces dereferencing: automated parameter capture for non-character pointer types (`T*`) strictly dereferences the element at index zero via the `ReadPointer<T>` helper. If an instrumentation policy requires inspecting across array elements, this logic cannot be expressed through automated YAML telemetry rules. Instead, buffer inspection is explicitly delegated to user-defined pre- or post-execution callbacks implemented within the companion C++ logic file (Section 3.2.2). Inside these callbacks, the developer operates with full C++ expressiveness, enabling array traversal.

3.5.3 Recursion Guard

In DBI, a common cause of unexpected process termination is *hook re-entrancy*. Because hooks are installed globally at the entry point of target APIs, any code executed inside the hook payload that directly or indirectly calls a hooked function can trigger an infinite loop on the calling thread.

3.5.3.1 The Re-Entrancy Mechanism

Let's consider an instrumentation configuration where the `WriteFile` API is hooked. When the host application invokes `WriteFile`, execution jumps to the hook function. Inside the

function, a logging rule or diagnostic trace might call a utility that writes a debug string to a file or invokes an internal logging routine that ultimately calls `WriteFile`. Without re-entrancy protection, this second call re-enters the hook function from the beginning, which again triggers the logger, forming an infinite recursion loop:

$$\text{HookedWriteFile} \longrightarrow \text{Log} \longrightarrow \text{WriteFile} \longrightarrow \text{HookedWriteFile} \longrightarrow \ldots$$

Because each nested invocation allocates a new stack frame, the thread stack is rapidly consumed. Stack overflow generates a `STACK_OVERFLOW` exception, terminating the host application.

3.5.3.2 The RecursionHookGuard Definition

HookDesigner eliminates the re-entrancy risk through a per-thread guard. During hook payload initialization (`InitializeHooks`), the framework allocates a process-wide TLS slot index:

```
static DWORD g_TlsIndex = TLS_OUT_OF_INDEXES;
// Inside InitializeHooks():
g_TlsIndex = TlsAlloc();
```

Using this slot, HookDesigner implements a RAII helper structure named `RecursionHookGuard` in the helper file `_user_logic_helpers.cpp`, shown in code 3.16:

Code 3.16: `RecursionHookGuard` RAII implementation in `_user_logic_helpers.cpp`.

```
struct RecursionHookGuard {
    bool active;
    RecursionHookGuard() : active(false) { Acquire(); }
    ~RecursionHookGuard() { Release(); }

    void Acquire() {
        if (!active) {
            TlsSetValue(g_TlsIndex, (LPVOID)1);
            active = true;
        }
    }

    void Release() {
        if (active) {
            TlsSetValue(g_TlsIndex, (LPVOID)0);
            active = false;
        }
    }
};
```

3.5.3.3 The New Hook Function Control Flow

As shown in Figure 3.3, at the entry of each generated hook function (`Hooked<F>`), before executing any code, the function inspects the thread's recursion state:

```
if (TlsGetValue(g_TlsIndex) != 0) {
    return True<FunctionName>(param1, param2, ...);
}
```

If the thread-local storage (TLS) value is non-zero, the thread is already executing inside an active hook function. The function immediately redirect execution directly to the original trampoline pointer (`True<FunctionName>`), bypassing all hook logic.

If the thread is not in a recursive call (`TlsGetValue == 0`), the function instantiates a local `RecursionHookGuard` on the stack, which sets the TLS flag to `1` (`Acquire()`). Crucially, right before invoking the trampoline to execute the original target function, the function calls:

```
hook_recursion_guard.Release();
```

This pre-trampoline release is a deliberate choice:

- While user pre-hook and post-hook injected logic execute, the guard is *active* (`1`), blocking any re-entrant hook loops triggered by callbacks or telemetry code.
- While the original target function executes, the guard is *released* (`0`). If the target function legitimately calls *another* hooked function as part of its normal operation, that nested call is permitted to enter its own hook function and be monitored.

After the trampoline returns, the function re-acquires the guard (`hook_recursion_guard.Acquire()`) to protect post-hook execution and telemetry recording.

Finally, when the hook function completes and returns, `hook_recursion_guard` goes out of scope on the stack. Its RAII destructor automatically invokes `Release()`, restoring the TLS slot to `0`. By relying on RAII, even if an unexpected exception occurs at any point during pre-hook or post-hook execution, the stack unwinding mechanism automatically disarms the guard, ensuring the thread is always returned to a fair, non-recursive state.

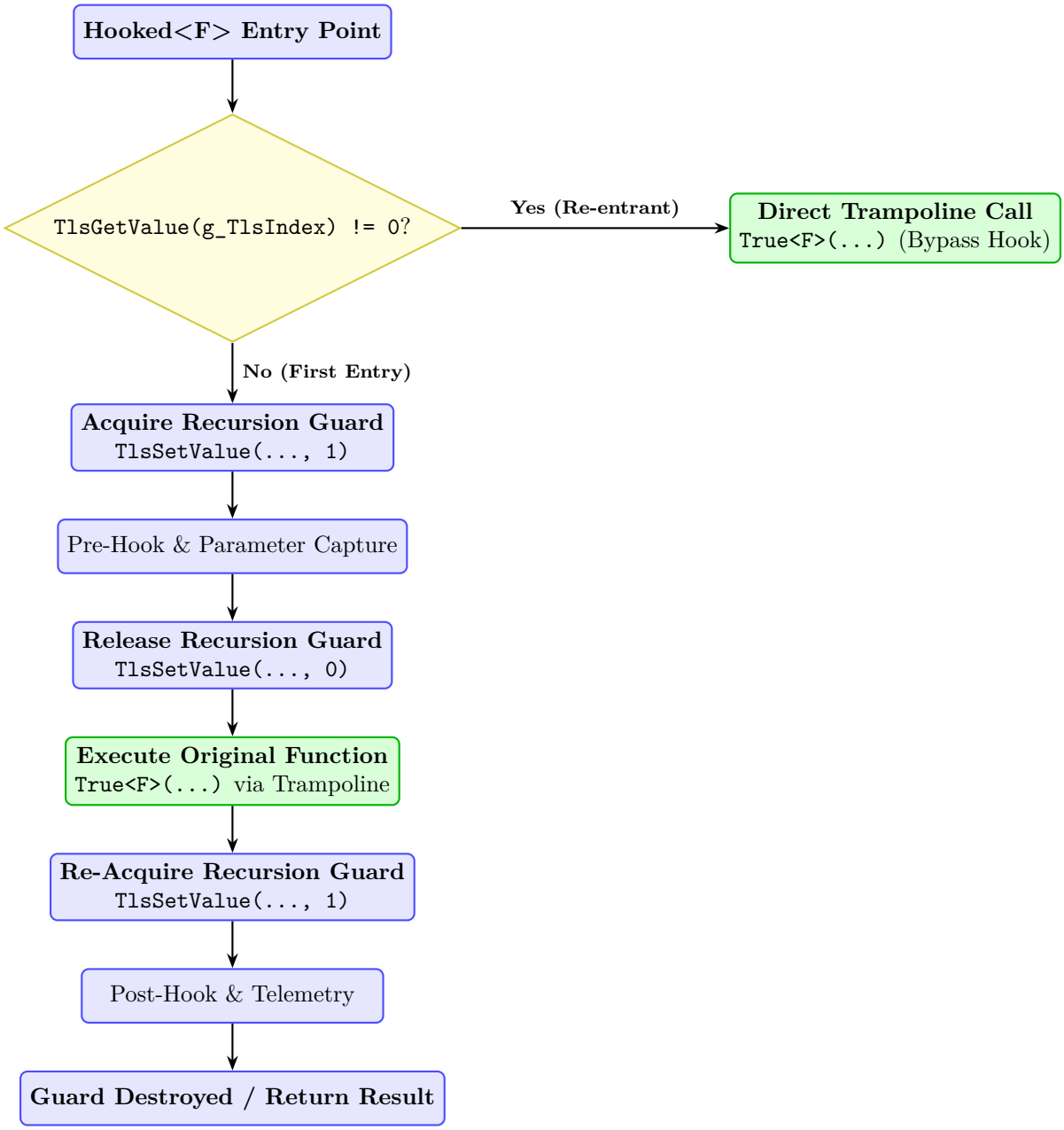


Figure 3.3: Hook control flow with TLS-based recursion guard

3.5.4 The Complete Hook Function Structure

All the presented Hook safety features converge in the implementation of the hook function body (`_hook_body.cpp`). Depending on whether the intercepted target function returns

to its caller or is a non-returning function, HookDesigner generates one of two distinct structures:

3.5.4.1 Standard Hook Function Body

To illustrate how these defenses integrate in practice for standard returning targets, code 3.17 presents a complete generated C++ Hook function.

Code 3.17: Complete generated hook function demonstrating integrated safety mechanisms and decoupled exception boundaries.

```
int __stdcall HookedProcessPacket(int packetId, char* payload, double* sensorVal) {
    // Thread Re-Entrancy Check
    if (TlsGetValue(g_TlsIndex) != 0) {
        return TrueProcessPacket(packetId, payload, sensorVal);
    }

    // Thread Guards (TLS Recursion & Active Thread Tracking)
    RecursionHookGuard hook_recursion_guard;
    HookActiveThreads hook_active_threads;

    // 1. Entry Parameter Capture Boundary
    bool telemetry_inputs_ready = false;
    const char* capturing_input_param = nullptr;
    int input_packetId = {};
    std::string input_payload = {};
    double input_sensorVal = {};
    try {
        capturing_input_param = "packetId";
        input_packetId = packetId;
        capturing_input_param = "payload";
        input_payload = CopyString(payload);
        capturing_input_param = "sensorVal";
        input_sensorVal = ReadPointer(sensorVal);
        telemetry_inputs_ready = true;
    } catch (const std::exception& ex) {
        OutputDebugStringA(fmt::format("[HOOK CAPTURE] Exception in {}: {}\n",
    capturing_input_param, ex.what()).c_str());
    } catch (...) {
        OutputDebugStringA(fmt::format("[HOOK CAPTURE] Hardware/SEH fault in {}\n",
    capturing_input_param).c_str());
    }

    // 2. Pre-Logic Execution Boundary
    try {
        ProcessPacket_pre(packetId, payload);
    } catch (const std::exception& ex) {
        OutputDebugStringA("[HOOK EXCEPTION] Pre-hook error in ProcessPacket\n");
```

```
    } catch (...) {
        OutputDebugStringA("[HOOK EXCEPTION] Access violation in pre-hook\n");
    }

    LARGE_INTEGER start_time;
    QueryPerformanceCounter(&start_time);

    // Trampoline Execution (Guard Temporarily Disarmed)
    hook_recursion_guard.Release();
    int __hook_result = TrueProcessPacket(packetId, payload, sensorVal);

    LARGE_INTEGER end_time;
    QueryPerformanceCounter(&end_time);
    double elapsed_ms = (end_time.QuadPart - start_time.QuadPart) * 1000.0 /
    g_PerfFrequency.QuadPart;

    // After Trampoline execution, re-acquire guard
    hook_recursion_guard.Acquire();

    // 3. Post-Logic Execution Boundary
    try {
        ProcessPacket_post(__hook_result, payload);
    } catch (const std::exception& ex) {
        OutputDebugStringA("[HOOK EXCEPTION] Post-hook error in ProcessPacket\n");
    } catch (...) {
        OutputDebugStringA("[HOOK EXCEPTION] Access violation in post-hook\n");
    }

    // 4. Exit Parameter Capture Boundary
    bool telemetry_outputs_ready = false;
    const char* capturing_output_param = nullptr;
    double output_sensorVal = {};
    try {
        capturing_output_param = "sensorVal";
        output_sensorVal = ReadPointer(sensorVal);
        telemetry_outputs_ready = true;
    } catch (const std::exception& ex) {
        OutputDebugStringA(fmt::format("[HOOK CAPTURE] Exception in output {}: {}\n",
    capturing_output_param, ex.what()).c_str());
    } catch (...) {
        OutputDebugStringA(fmt::format("[HOOK CAPTURE] Hardware/SEH fault in output {}\
    n", capturing_output_param).c_str());
    }

    // 5. Guarded Telemetry Evaluation Boundary
    if (telemetry_inputs_ready && telemetry_outputs_ready) {
        try {
            spdlog::info("[PACKET] ID: {}, Latency: {:.2f}ms", input_packetId,
    elapsed_ms);
```

```
            if (g_PromMetric_ProcessPacket_invocations) {
                g_PromMetric_ProcessPacket_invocations->Increment();
            }
        } catch (const std::exception& ex) {
            OutputDebugStringA("[TELEMETRY EXCEPTION] Exception in telemetry\n");
        } catch (...) {
            OutputDebugStringA("[TELEMETRY EXCEPTION] Unknown exception in telemetry\n"
    );
        }
    }

    return __hook_result;
}
```

3.5.4.2 Adaptation for Non-Returning Targets

The standard hook function body operates on the premise that the intercepted target function eventually returns after execution. However, several system APIs are *non-returning functions*: they terminate the thread or process, or transfer control directly to an external exception dispatcher, never returning control to the caller.

If a non-returning function is instrumented using the standard returning function structure, the instrumentation payload encounters the following architectural failures:

1. **Post-Trampoline Telemetry dead-code:** In _hook_body.cpp, telemetry evaluation is executed after the trampoline call inside the post-execution block, for this reason it is never reached with this type of functions
2. **Bypassed C++ Stack Unwinding:** When a process or thread terminates, the operating system immediately tears down the user-mode execution context. Standard C++ stack unwinding is skipped, meaning that local RAII destructors on the stack are *never executed*. Consequently, the destructor of `HookActiveThreads` (code 2.10) never runs, causing the global `activeThreads` counter to remain permanently greater than zero. During payload ejection, this causes the injector to believe threads are permanently trapped in the hook, triggering an unnecessary DLL leak.

HookDesigner resolves these issues through automated adaptations applied to the standard hook function body:

1. **Pre-Trampoline Telemetry:** In `_hook_body.cpp`, telemetry evaluation is shifted to execute *before* the trampoline call inside the pre-execution block (code 3.18).

2. **Targeted Thread Tracking and Explicit Guard Destruction:** Since process termination skips local destructors, the function explicitly calls `hook_active_threads.Release()` method deifned in the final version of the guard (Section 3.6.3.1) immediately before invoking the terminal trampoline (code 3.18). This decrements the global `activeThreads` counter in advance, ensuring DLL ejection synchronization safety (Section 3.6.3). For applying this solution also in Frida, the framework substitutes `gum_interceptor_replace` with the raw replacement API (`gum_interceptor_replace_fast`, Section 2.4.2.2), substituting the automatic Frida-Gum thread-tracking mechanism (Section 2.4.3.1) with the manual `HookActiveThreads` RAII guard *only* for non-returning functions (unlike Microsoft Detours, which requires it universally for all hooks).

The resulting hook function structure implementing these mitigations is presented in code 3.18.

Code 3.18: Generated hook function structure for non-returning functions

```
void WINAPI HookedRaiseException(DWORD dwExceptionCode, DWORD dwExceptionFlags,
                                 DWORD nNumberOfArguments, const ULONG_PTR* lpArguments
    )
{
    if (TlsGetValue(g_TlsIndex) != 0) {
        TrueRaiseException(dwExceptionCode, dwExceptionFlags, nNumberOfArguments,
    lpArguments);
        return;
    }

    RecursionHookGuard hook_recursion_guard;
    HookActiveThreads hook_active_threads;

    // 1. Entry Parameter Capture Boundary
    bool telemetry_inputs_ready = false;
    const char* capturing_input_param = nullptr;
    DWORD input_dwExceptionCode = {};
    DWORD input_dwExceptionFlags = {};
    try {
        capturing_input_param = "dwExceptionCode";
        input_dwExceptionCode = dwExceptionCode;
        capturing_input_param = "dwExceptionFlags";
        input_dwExceptionFlags = dwExceptionFlags;
        telemetry_inputs_ready = true;
    } catch (...) {
        OutputDebugStringA("[HOOK CAPTURE] Fault capturing input parameters\n");
    }

    // 2. Pre-Hook User Logic Boundary
    try {
```

```
        RaiseException_pre(dwExceptionCode, dwExceptionFlags, nNumberOfArguments);
    } catch (...) {
        OutputDebugStringA("[HOOK EXCEPTION] Exception in RaiseException pre-hook\n");
    }

    // 3. Guarded Inverted Telemetry (BEFORE Trampoline)
    if (telemetry_inputs_ready) {
        try {
            spdlog::error("[RaiseException] Code: 0x{:x}, Flags: 0x{:x}",
                          input_dwExceptionCode, input_dwExceptionFlags);
        } catch (...) {
            OutputDebugStringA("[TELEMETRY EXCEPTION] Exception in telemetry\n");
        }
    }

    // EXPLICIT GUARD RELEASE: Decrements activeThreads before trampoline
    hook_recursion_guard.Release();
    hook_active_threads.Release();

    // Trampoline Invocation (Never returns)
    TrueRaiseException(dwExceptionCode, dwExceptionFlags, nNumberOfArguments,
    lpArguments);
}
```

3.6 DLL Lifecycle and Deployment

After the offline compilation stage (Stage 4, Section 3.1.2) completes, the instrumentation artifact exists as a self-contained Windows DLL. To monitor and manipulate the host application, this payload must be moved to the online context: it must be mapped into the target process's virtual address space, activate its hooks, execute safely alongside application, and cleanly detach when monitoring terminates.

This section examines the deployment and runtime lifecycle of HookDesigner payloads, managed by the standalone injector utility (`injector.exe`, implemented in `injector.cpp`). Specifically, Section 3.6.1 details the remote injection process, Section 3.6.2 presents the suspended-process launch mode, designed for launching the process already containing the payload instead of injecting it in a running instance, and Section 3.6.3 analyzes the ejection process.

3.6.1 DLL Injection

In all the modern operating systems, address space isolation (Section 2.2.2) prevents one process from directly manipulating the execution state of another. To introduce the compiled hook payload into a running target, HookDesigner implements dynamic library injection via the standard Windows debugging and threading APIs.

3.6.1.1 The DLL Injection Process

To attach to a running target process identified by its process identifier (PID), the standalone injector utility (`injector.exe`) leverages the classical remote thread injection technique introduced in Section 2.2.3. After obtaining a process handle via `OpenProcess`, the injector writes the absolute path of the hook DLL into the target's virtual address space using `VirtualAllocEx` and `WriteProcessMemory`.

To execute `LoadLibraryA` remotely, the injector exploits an architectural property of the Windows subsystem: core system libraries (specifically `Kernel32.dll`) are mapped to identical virtual base addresses across all processes on a given system boot. Consequently, the address of `LoadLibraryA` in the injector's address space is identical in the target process. The injector spawns a remote thread pointing to `LoadLibraryA` with the path buffer as its parameter via `CreateRemoteThread`, and waits for thread completion. Once the remote thread finishes and the memory buffer is released, the DLL is mapped into the target process.

3.6.1.2 Relative Virtual Address Offset Invariance

Once `LoadLibraryA` returns, the hook DLL is loaded in the target process, but its hooks are still disabled. The injector must now remotely invoke the module's exported initialization function, `InitializeHooks`. However, under Address Space Layout Randomization (ASLR), the operating system loader maps the hook DLL to an arbitrary base address in the target process that differs from where it might be loaded in the injector.

To resolve the address of exported remote functions across ASLR boundaries, `injector.cpp` exploits the property of *RVA offset invariance* within Portable Executable (PE) binaries via the `CallRemoteExport()` helper (code 3.19):

1. **Local DLL Loading:** The injector loads the hook DLL locally with `LoadLibraryExA` specifying `DONT_RESOLVE_DLL_REFERENCES`. This flag instructs the Windows loader to map the binary and its PE headers into memory without invoking `DllMain` and without resolving external dependencies, avoiding useless side effects.

2. **RVA Offset Calculation:** The injector queries the exported function's local virtual address and computes its offset relative to the local module base address. Because the relative layout of code sections and the export directory is fixed at link time and does not change when a module is relocated, this offset is *invariant* across all address spaces.

3. **Remote Base Address Discovery:** The injector enumerates the target process's loaded modules using the Toolhelp32 snapshot API (`CreateToolhelp32Snapshot`, locating the target's remote base address `hRemoteModule` matching the DLL filename.

4. **Remote Function Execution:** The remote function address is computed by adding the offset relative to the remote base. After freeing the temporary local module mapping with `FreeLibrary`, the injector launches a remote thread targeting $pRemoteFunc$ and waits for it to complete.

Code 3.19: Cross-process export resolution via RVA offset invariance in `CallRemoteExport`.

```
// 1. Map DLL locally without side-effects
HMODULE hLocalMod = LoadLibraryExA(localDllPath.c_str(), NULL,
    DONT_RESOLVE_DLL_REFERENCES);
FARPROC pLocalFunc = GetProcAddress(hLocalMod, exportName.c_str());

// 2. Compute invariant RVA offset and resolve target address under ASLR
DWORD_PTR offset = (DWORD_PTR)pLocalFunc - (DWORD_PTR)hLocalMod;
LPVOID pRemoteFunc = (LPVOID)((DWORD_PTR)hRemoteModule + offset);
FreeLibrary(hLocalMod);

// 3. Trigger remote execution in target process
HANDLE hThread = CreateRemoteThread(hProcess, NULL, 0,
    (LPTHREAD_START_ROUTINE)pRemoteFunc, NULL, 0, NULL);
WaitForSingleObject(hThread, INFINITE);
```

3.6.1.3 The Loader Lock Issue

A common anti-pattern in DBI is to perform complex tasks directly inside the DLL entry point, `DllMain`, as introduced in code 2.1. While convenient for trivial payloads, executing complex instrumentation routines inside `DllMain` introduces severe concurrency issues on Windows due to the OS *Loader Lock*.

Whenever the Windows operating system maps or unmaps a DLL, or when a process creates or terminates a thread, the OS loader must acquire an internal, process-wide synchronization primitive known as the Loader Lock. While this lock is held, the loader

sequentially executes the `DllMain` routines of all loaded modules. Consequently, binary engineering guidelines explicitly prohibit thread synchronization, waiting on external events, and socket initialization within `DllMain` [Mic26d].

HookDesigner's telemetry (whose initialization would be in `DLL_PROCESS_ATTACH`:) (Section 3.4) relies on two asynchronous background subsystems:

- **Asynchronous Logging** (`spdlog`): Spawns a background worker thread to drain ring buffers and flush log messages.
- **Metrics Server** (`prometheus-cpp`): Spawns CivetWeb HTTP listener threads to serve metric scrapers.

A circular deadlock occurs whenever the calling thread inside `DllMain` blocks waiting upon those background threads. In HookDesigner's architecture, executing lifecycle logic under Loader Lock exposes two catastrophic deadlock modes:

1. **The Initialization Rollback Deadlock:** During normal startup, telemetry initialization requires asynchronous thread creation without an immediate startup join. However, if any subsequent initialization step fails, the exception handler immediately invokes the rollback sequence (code 3.14) to prevent partial instrumentation. This cleanup triggers a blocking call (`WaitForSingleObject`) on the newly spawned worker threads. Because those workers are permanently trapped waiting for the Loader Lock held by `DllMain` and the thread in `DllMain` waits on those worker threads, the result is a circular deadlock.
2. **The Deinitialization Deadlock:** Similarly, when an instrumentation payload is dynamically unloaded, the operating system acquires the Loader Lock before invoking `DllMain(DLL_PROCESS_DETACH)`. Executing `DeinitializeHooks()` inside this handler requires tearing down and joining all background telemetry threads before the module is unmapped from memory. As in the rollback scenario, joining these exiting threads while holding the Loader Lock triggers the same circular deadlock seen in the rollback mechanism.

3.6.1.4 The Two-Phase Protocol

To eliminate this issue, HookDesigner separates DLL mapping from hooks activation using a *Two-Phase Protocol*:

- **Phase 1 (Passive Mapping):** The injector invokes `CreateRemoteThread` targeting `LoadLibraryA`. Inside the generated payload (`_dllmain.cpp`), `DllMain` executes under Loader Lock but performs only minimal operations:

```
BOOL APIENTRY DllMain(HMODULE hModule, DWORD ul_reason_for_call, LPVOID lpReserved
    ) {
    if (ul_reason_for_call == DLL_PROCESS_ATTACH) {
        DisableThreadLibraryCalls(hModule);
    }
    return TRUE;
}
```

Calling `DisableThreadLibraryCalls` informs the Windows Loader that the DLL does not require notifications when application worker threads are created or terminated (`DLL_THREAD_ATTACH` or `DLL_THREAD_DETACH`). This eliminates thread-notification overhead during application runtime. Crucially, `DllMain` creates no threads, makes no network calls. It returns `TRUE` immediately, allowing the OS loader to release Loader Lock cleanly.

- **Phase 2 (Active Initialization):** Once the Phase 1 remote thread exits, the injector computes the remote address of `InitializeHooks` via RVA offset invariance (code 3.19) and launches a second remote thread. Because this second thread runs entirely outside `DllMain` and so outside Loader Lock, it can safely initialize the `spdlog` background thread pool, bind the Prometheus HTTP metrics server, and install transactional inline hooks, bypassing the Loader Lock deadlock risk.

Furthermore, the injector inspects the remote exit code returned by `InitializeHooks`. In injection mode, if initialization fails (returning a non-zero code), the injector automatically executes a rollback by remotely invoking `FreeLibrary` to unload the DLL, leaving the host process running uncorrupted.

3.6.1.5 Engine-Specific Initialization Mechanics

While Section 3.3.5 introduced the simplified template structure used to emit the `InitializeHooks()` skeleton (codes 3.7 and 3.8), the actual binary payload executes a more complex initialization process during Phase 2. This process incorporates all the HookDesigner's features we have seen so far throughout Sections 3.4 and 3.5, adapting the attachment strategy to the underlying engine as summarized in Table 3.10.

Initialization Step	Microsoft Detours	Frida-Gum
Recursion Safety	Allocates TLS index (`TlsAlloc`)	Allocates TLS index (`TlsAlloc`)
Telemetry Engine	Initialized via `_telemetry_init.cpp` (`spdlog` thread pool, QPC frequency, Prometheus Exposer)	Identical initialization via shared partial `_telemetry_init.cpp`
Runtime Core	None (Direct binary patching)	Calls `gum_init_embedded()` and `gum_interceptor_obtain()`
Address Resolution	Evaluates `GetProcAddress` or base + RVA offset	Evaluates `GetProcAddress` or base + RVA offset
Thread Enumeration	Calls `EnlistAllThreads()` via Toolhelp32 snapshot	None
Transactional Scope	`DetourTransactionBegin()`	`gum_interceptor_begin_transaction()`
Thread Enlistment	Calls `DetourUpdateThread()` for all worker threads	None
Hook Attachment	`DetourAttach(&(PVOID&)True<F>, Hooked<F>)`	Calls `gum_interceptor_replace()` (standard) or `replace_fast()` (non-returning)
Transaction Commit	`DetourTransactionCommit()` (suspends threads, relocates IP, applies patch)	`gum_interceptor_end_transaction()` (atomic instruction write)

Table 3.10: Comparison of Hooks initialization between Microsoft Detours and Frida-Gum.

3.6.2 Injector Launch Mode

Dynamic injection into an active process (`injector.exe inject <PID> <DLL>`) is appropriate when instrumenting long-running processes. However, this approach introduces a problem: any operations performed by the target before the injector attaches are missed. To solve this problem, `injector.cpp` provides a *launch mode* designed for launching the process already containing the instrumentation payload; it can be invoked as:

```
injector.exe launch <Executable_Path> <DLL_Path> [optional_target_args...]
```

3.6.2.1 Suspended-Process Mechanism

The execution sequence for launch mode proceeds through four steps:

1. **Process Instantiation:** The utility calls `CreateProcessA` specifying the process creation flag `CREATE_SUSPENDED`. When launching a process with this flag, the primary thread is created and suspended before reaching the application entry point [Yos+17].

   Consequently, when `InitializeHooks` is invoked via a remote thread prior to resuming the primary thread, `GetModuleHandleA` can only resolve modules present in the static Import Address Table (IAT) or those imported by the injected DLL itself. Any library that the target application loads dynamically at runtime via `LoadLibrary` or through delay-loading (`/DELAYLOAD`) will not yet be present in the PEB loader database; resolving symbols within such modules yields `NULL`, causing the hook initialization routine to log a warning and skip attachment for those targets.

2. **Hook Activation:** While the main thread remains frozen, the injector applies the Two-Phase Protocol (Section 3.6.1.4):

   - It injects the instrumentation DLL via a remote thread pointing to `LoadLibraryA`.
   - It computes the remote address of `InitializeHooks` and activates all hooks via a second remote thread.

   Upon completion of `InitializeHooks`, the injector validates the remote thread's exit code: if it is non-zero (indicating Hooks initialization failure), the injector aborts and terminates the suspended process via `TerminateProcess` to avoid executing an uninstrumented or partially configured target.

3. **Primary Thread Resumption:** When `InitializeHooks` returns `0` (success), the injector calls `ResumeThread`, decrementing the suspend count to zero. The primary thread awakens and begins executing its startup sequence. Because hooks are already active, any function intercepted by the policy that executes during application startup is captured from its very first call.

4. **Process Termination:** The injector observes process execution, queries its exit code via `GetExitCodeProcess`, and cleanly terminates.

Figure 3.4 compares the dynamic injection pipeline with the suspended launch pipeline, illustrating how both share the Two-Phase protocol while targeting different process states and lifecycle requirements.

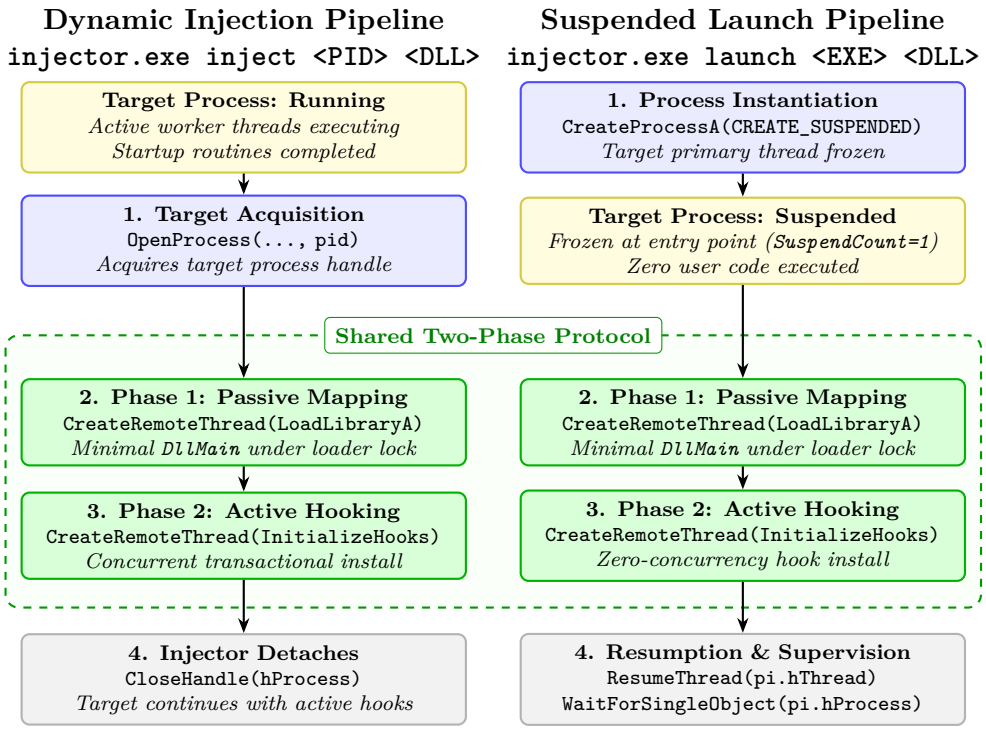


Figure 3.4: Architectural comparison between Dynamic Injection mode and Suspended Launch mode

3.6.3 DLL Ejection

When instrumentation is no longer required, the payload must detach cleanly so that the target process can continue running without interruption. As analyzed in Section 2.4.6, reversing prologue bytes is necessary but not sufficient: in a multi-threaded application, worker threads may already be executing within hook routines when detachment begins. If the host unmaps the DLL via `FreeLibrary` while a worker thread's instruction pointer points to unmapped memory, the process crashes immediately with an Access Violation.

To prevent this, HookDesigner uses a thread-tracking mechanism during hook detachment.

3.6.3.1 Hook Active Thread Tracking

In generated hook functions, thread execution scope is tracked using a RAII helper `HookActiveThreads`, defined in `_user_logic_helpers.cpp` and shown in code 3.20, similar to the one introduced in Section 2.4.6.

Code 3.20: HookDesigner implementation of active thread tracking in `_user_logic_helpers.cpp`.

```
static std::atomic<int> activeThreads{0};

// RAII helper to track active hook execution count
struct HookActiveThreads {
    bool active;
    HookActiveThreads() : active(true) {
        activeThreads++;
    }
    ~HookActiveThreads() {
        if (active) activeThreads--;
    }
    void Release() {
        if (active) {
            activeThreads--;
            active = false;
        }
    }
};
```

- **Lock-Free Counter:** The counter `activeThreads` is a global atomic variable (`std::atomic<int>`). Its constructor uses atomic increment (`activeThreads++`) on entry, and its destructor executes atomic decrement (`activeThreads-`) on exit.
- **Explicit Guard Release for Terminating APIs:** As detailed in Section 3.5.4.2, functions like `ExitProcess` or `RaiseException` never return, so stack unwinding and local destructors are bypassed. For this reason I have added the explicit Guard Release method, that the hook explicitly calls `hook_active_threads.Release()` immediately before invoking the terminal trampoline, ensuring the atomic counter decrements before the thread terminates.

3.6.3.2 The DLL Ejection Mechanism

When `injector.exe eject <PID> <DLL>` is executed, it remotely invokes the exported function `DeinitializeHooks()` via `CallRemoteExport`, using the same method of the injection, (code 3.19), for preventing Loader Lock deadlock (Section 3.6.1.3) possible also during ejection. The payload then performs the following five-step ejection sequence:

1. **Terminate Background Listener Threads:** The payload resets the Prometheus HTTP exposer. The underlying CivetWeb server runs internal threads blocked in `select()` or `accept()` calls; destroying the exposer closes the listening socket and

terminates these web worker threads cleanly. This step must occur first so that external HTTP scraping requests do not arrive while metric families and registries are being destroyed.

2. **Atomically Revert Hook Patches:** The payload detaches all inline hooks using the engine's transactional API, detailed in Section 2.4.5 Once the transaction commits, original prologue bytes are restored in memory. New function calls execute the unhooked target code directly; no new threads can enter the hook functions.

3. **Bounded Drain Wait:** Although new calls cannot enter, worker threads that were already inside the hook routines before Step 2 may still be executing. To synchronize safely, the payload polls a drain condition for at most $T_{\text{drain}} = 2\,\text{s}$ (Algorithm 2). What "drained" means depends on the engine and, for Frida-Gum, on the policy (Table 3.11); the template fixes this choice once, at code-generation time.

   - Under Detours, every hook increments the `activeThreads` counter, so the drain condition is that the counter reaches zero.
   - Under Frida-Gum with only returning hooks, the interceptor tracks in-flight invocations itself, and `gum_interceptor_flush()` returning `TRUE` (no thread is still inside the interceptor's trampolines) is sufficient.
   - Under Frida-Gum with at least one non-returning hook, those hooks bypass the interceptor's tracking (Section 3.5.4.2), so both conditions are required.

   Whenever the counter takes part in the condition, a *confirmation interval* T_{confirm} follows before the module may be considered free. The `HookActiveThreads` destructor decrements the counter just before the hook function returns, so a thread can read zero while still executing the epilogue (register restoration and the final `ret`) inside the DLL's pages. Unmapping at that instant would fault a preempted thread, so the counter is re-read after T_{confirm}. This is a heuristic, not a proof: a thread preempted for longer than T_{confirm} at exactly that point would go undetected. A second call after a failed drain skips the revert (the hooks are already detached) and re-runs only the drain phase, which is what makes the repeated ejection of Section 3.6.3.3 possible.

4. **Destruction of Telemetry Subsystems:** Only after Step 3 confirms that all worker threads have drained (`g_Ejected == true`) the payload destroy the logging and metrics registries. This ordering is critical: if loggers or registries were destroyed while a worker thread was still inside `_telemetry_body.cpp` formatting a log or incrementing a counter, the thread would encounter destroyed pointers, triggering a use-after-free crash.

5. **Runtime and Resource Deallocation:** The payload releases the TLS slot. Under Frida-Gum, it unreferences the interceptor and shuts down embedded Frida runtime

engine. Finally, `DeinitializeHooks()` returns `0` to signal a clean drain and safe detachment to the injector, or `1` if active threads remain trapped.

Generated for	**Drained Condition**	***needConfirm***
Detours	`activeThreads` $= 0$	yes
Frida-Gum, returning hooks only	`gum_interceptor_flush()`	no
Frida-Gum, ≥ 1 non-returning hook	`gum_interceptor_flush()` $\wedge$ `activeThreads` $= 0$	yes

Table 3.11: Drain condition for each engine and policy.

3.6.3.3 Ejection-Failure DLL Leak Strategy

Despite all the safety measures adopted in the ejection protocol, a persistent challenge in DBI is the problem of *blocked worker threads*.

3.6.3.3.1 The Problem of Hooking Blocking Functions In industrial and network-connected applications, worker threads frequently execute blocking system calls, such as:

- `recv()` or `select()` waiting for incoming network packets on industrial buses;
- `WaitForSingleObject()` or `Sleep()` waiting for timer ticks or state updates;
- `GetMessage()` or `ReadFile()` waiting for user input or file I/O completion.

If an instrumentation hook intercepts a function that performs a blocking call, a worker thread may enter the hook function, call the original function via the trampoline, and remain blocked in the kernel for seconds, minutes, or hours. When the user initiates module ejection, Step 2 of `DeinitializeHooks()` (Algorithm 2) successfully restores the original prologue bytes. However, the blocked thread's activation frame remains frozen on its stack: its return address points directly into `Hooked<F>` inside the hook DLL.

If the injector unloads the DLL via `FreeLibrary`, the operating system unmaps the DLL's code pages from the process's virtual address space. When the blocking call eventually returns, the worker thread attempts to resume execution at an address that is no longer mapped, triggering an unhandled Access Violation that crashes the host application.

3.6.3.3.2 The Controlled Leak Mechanism To prevent this event, HookDesigner implements a defense coordinated between the injected DLL and the injector utility, illustrated in Figure 3.5:

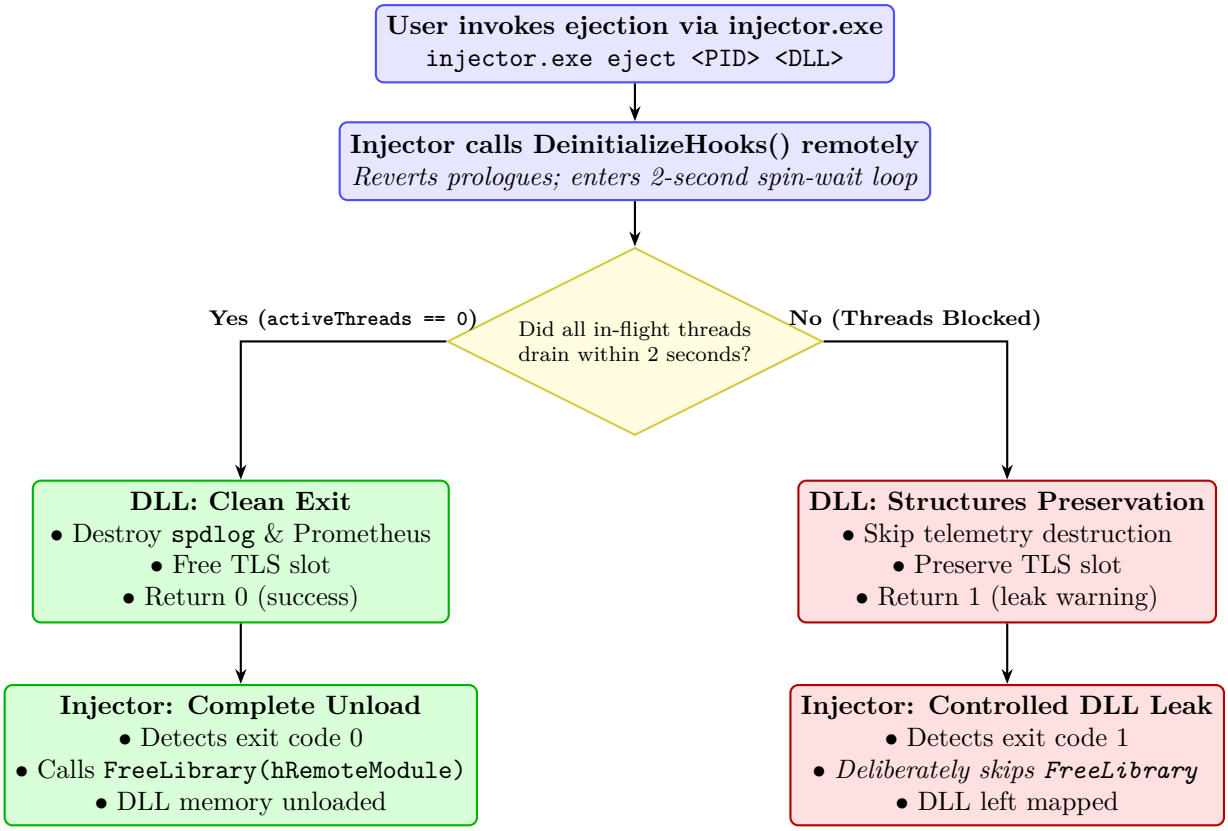


Figure 3.5: The Ejection-Failure DLL Leak Strategy

1. **DLL – Data Structures Preservation (`DeinitializeHooks`):** When the 2-second spin-wait loop expires and threads remain trapped `DeinitializeHooks()` recognizes that clean ejection is impossible. Instead of proceeding with standard cleanup, it:

   - **Preserves Telemetry Registries:** It deliberately skips telemetry data destruction. Leaving these structures intact ensures that when the blocked worker thread eventually wakes up and executes post-trampoline telemetry logic, it will not crash due to null pointer dereferences.
   - **Preserves TLS Index:** It deliberately skips releasing the allocated TLS slot: when a blocked worker thread unblocks and returns from the target API, it re-enters the hook function's post-logic, calling `hook_recursion_`

`guard.Acquire()` and subsequently `Release()` inside its RAII destructor (Section 3.5.3). Both routines call `TlsSetValue` using `g_TlsIndex`, if `TlsFree` had been invoked, `g_TlsIndex` would become invalid. Preserving `g_TlsIndex` guarantees that post-execution re-entrancy tracking and RAII destructors complete safely without process corruption.

- **Returns Status Code:** It returns `1` (thread leak warning code) to the injector, signaling that hooks have been detached but threads remain trapped in hook routines.

2. **Injector – Safety Fallback:** The injector inspects the remote thread's exit code, as shown in code 3.21:

Code 3.21: Injector fallback logic in `injector.cpp`.

```
DWORD deinitExitCode = (DWORD)-1;

CallRemoteExport(hProcess, hRemoteModule, dllPath, "DeinitializeHooks", &
    deinitExitCode);

if (deinitExitCode == 1) { // trapped threads warning: DLL leak  fallback
    std::cout << "[WARNING] Hooks successfully detached, but some threads remain
    blocked inside them.\n";
    std::cout << "[WARNING] Skipping FreeLibrary (leaking DLL in target memory) to
     prevent target crash.\n";
} else if (deinitExitCode != 0) { // Detachment error
    std::cerr << "[ERROR] DeinitializeHooks reported error code: 0x"
              << std::hex << deinitExitCode << ". Skipping FreeLibrary.\n";
} else { // Exit code 0: Clean drain confirmed
    std::cout << "Ejecting DLL...\n";
    HANDLE hThread = CreateRemoteThread(hProcess, NULL, 0,
        (LPTHREAD_START_ROUTINE)pFreeLibrary, (LPVOID)hRemoteModule, 0, NULL);
    WaitForSingleObject(hThread, INFINITE);
    CloseHandle(hThread);
}
```

If `deinitExitCode` is `1`, `injector.exe` outputs a warning and *deliberately skips* calling `FreeLibrary`. If `deinitExitCode` is any other non-zero value, an error occurred during detachment, and `FreeLibrary` is similarly bypassed to prevent unmapping unpredictable state. Only when `deinitExitCode == 0` (clean thread drain confirmed) does the injector safely invoke `FreeLibrary` to unmap the DLL.

From a software engineering perspective, leaking memory is typically considered a defect. However, in DBI—particularly in critical industrial software—leaking a DLL under ejection failure is a safety-oriented design decision.

Considering the two alternatives when ejection encounters blocked threads:

1. **Strict Memory Reclamation (`FreeLibrary`):** The injector unmaps the module. When the blocked thread wakes up, its return address points to deallocated memory (`MEM_FREE`), triggering an instantaneous Access Violation, that makes crash the host process.

2. **Controlled Payload Leak (HookDesigner Fallback):** The module remains mapped in virtual memory. Because hooks were already reverted in Step 2, no *new* calls can ever enter the DLL. The memory overhead of the leaked payload is ususally negligible. When the blocked thread eventually unblocks, it unwinds its stack through the still-valid code pages, returns to the host application, the ejection mechanism can be repeated and the DLL can be freed.

In critical environments, often *process stability supersedes complete memory reclamation.* By combining atomic prologue reversion with the controlled DLL leak fallback, HookDesigner guarantees that instrumentation can be safely detached at runtime without risking process termination.

Algorithm 2 `DeinitializeHooks`. `Drained` and `needConfirm` are fixed per engine and policy when the code is generated (Table 3.11); they are not evaluated per call.

Require: $T_{\text{drain}} = 2000\,\text{ms}$, $T_{\text{confirm}} = 1000\,\text{ms}$, $T_{\text{poll}} = 50\,\text{ms}$

if *ejected* **then**
    return TRUE
end if
if *hooksAttached* **then**
    stop the metrics exposer (if configured)
    revert all hooks in a single engine transaction
    if the transaction failed **then**
        return FALSE
    end if
    hooksAttached $\leftarrow$ FALSE
end if
$t_0 \leftarrow$ NOW()
while NOW() $- t_0 < T_{\text{drain}}$ **do**
    if DRAINED **then**
        if *needConfirm* **then**
            sleep T_{confirm}
        end if
        if not *needConfirm* **or** *activeThreads* $= 0$ **then**
            ejected $\leftarrow$ TRUE
            break
        end if
    end if
    sleep T_{poll}
end while
if *ejected* **then**
    destroy the metrics registry and shut down the logger (if configured)
    release the engine runtime (Frida-Gum only) and free the TLS slot
end if
return *ejected*

Chapter 4

Results

Here you will present the results of your work. Your skeptical reader is now asking themself "is this working?". You will show here to what extent it does. Be honest about the limitations: if your system doesn't work in some cases, you should say so (and explain where and why, if you can). Like the Methodology chapter, this one may end up being split into multiple chapters.

Keep in mind that this shouldn't be a mere dump of experimental results; you're rather *teaching* your reader how and to what extent you managed to solve the problem you described in the Introduction (Chapter 1). If you have additional results that may be useful but are not necessary to understand the points you're making (e.g., you evaluated your system on multiple datasets and the results all tell the same story), the place for them is in an appendix.

This chapter should have a lot of links to the methodology chapter(s), because you're evaluating the choices you made there. If you developed a system made of multiple parts, make sure that you test them separately and together, so that the reader can understand how important each part is.

This chapter is likely to be quite full of figures and tables. Try to make them as informative as possible (e.g., use multiple lines in the same plot if possible). Showing graphs effectively is a complex art; try to spend some time on it and ask for guidance from your advisor. Whenever you have a figure or a table, make sure that you refer to it in the text (after all, if it's not referred to in the text it means it has no part in the story you're telling, so it has no place in this Chapter). Always use the text (and the caption, if you can) to explain what you want the reader to understand from the figure.

If you are using LaTeX, it will automatically place figures, tables, algorithms, etc. somewhere in the text. These parts of the documents are called *floats* because their position

Column 1	Column 2	Column 3
aaa	2	3.42
bbb	505	6.00
ccc	8	901.02

Table 4.1: A table using the LaTeX `booktabs` package. Note there are no vertical rules. In case you want to do more fancy stuff (e.g., merging cells), check also the `multirow` and `multicol` packages. It's generally a good idea for readability to align text to the left and numbers to the right (use the same number of significant digits).

floats around depending on LaTeX's will. There are options to advise LaTeX about where floats should be placed. My advice is not to waste too much time on this and trust that they will end up in a good place. Generally, write the floats' code before you refer to them: they will never be placed before the place they appear in.

For Figures, generally use vectorial formats (e.g., PDF, SVG) where it makes sense if you can. They will result in higher-quality images that are in most cases also smaller, leading to shorter compiling times and a smaller resulting PDF.

Tables look better without vertical rules: an example is in Table 4.1.

4.1 Discussion

At the end of this chapter, take a step back and summarize all the results so that the reader can understand the big picture. Sometimes, this becomes large and important enough to be worth a chapter on its own.

Chapter 5

Conclusion

You are finally done! Here you will summarize for the reader what you have taught them through the document, and the main takeaways of your work you would like them to remember.

It is also a good idea to discuss the limitations of your work and your views about what can be possible future work.

Bibliography

[BGA03] Derek Bruening, Timothy Garnett, and Saman P. Amarasinghe. "An Infrastructure for Adaptive Dynamic Optimization". In: *1st International Symposium on Code Generation and Optimization (CGO 2003), 23-26 March 2003, San Francisco, CA, USA*. IEEE Computer Society, 2003, pp. 265–275. DOI: 10.1109/CGO.2003.1191551. URL: https://doi.org/10.1109/CGO.2003.1191551.

[Fri23] Frida Developers. *Frida Documentation: C API (Frida-Gum)*. 2023. URL: https://frida.re/docs/c-api/.

[Fri26] Frida Developers. *Frida-Gum: Cross-platform instrumentation and introspection library*. Version 17.9.3. 2026. URL: https://github.com/frida/frida-gum.

[HB99] Galen C. Hunt and Doug Brubacher. "Detours: Binary Interception of Win32 Functions". In: *Proceedings of the 3rd USENIX Windows NT Symposium, Seattle, WA, USA, July 12-13, 1999*. USENIX Association, 1999, pp. 135–143. URL: https://www.cs.columbia.edu/~junfeng/10fa-e6998/papers/detours.pdf.

[Int26] Intel Corporation. *Pin - A Dynamic Binary Instrumentation Tool*. Version 3.31. 2026. URL: https://software.intel.com/sites/landingpage/pintool/docs/99776/Pin/doc/html/index.html.

[Kag26] Tsuda Kageyu. *MinHook: The Minimalistic x86/x64 API Hooking Library for Windows*. Version 1.3.4. 2026. URL: https://github.com/TsudaKageyu/minhook.

[Lev00] John R. Levine. *Linkers and Loaders*. San Francisco, CA: Morgan Kaufmann, 2000. ISBN: 1-55860-496-0.

[LLV26] LLVM Developer Group. *Clang: A C Language Family Frontend for LLVM*. Version 18.1.1. 2026. URL: https://clang.llvm.org/.

[Luk+05] Chi-Keung Luk, Robert Cohn, Robert Muth, Harish Patil, Artur Klauser, Geoff Lowney, Steven Wallace, Vijay Janapa Reddi, and Kim Hazelwood. "Pin: building customized program analysis tools with dynamic instrumentation". In: *Proceedings of the 2005 ACM SIGPLAN Conference on Programming Language Design and Implementation, PLDI 2005, Chicago, IL, USA, June 12-15, 2005*. Ed. by Vivek Sarkar and Mary W. Hall. ACM, 2005, pp. 190–200. DOI: 10.1145/1064978.1065034. URL: https://doi.org/10.1145/1064978.1065034.

[Mel26] Gabi Melman. *spdlog: Very fast, header-only/compiled, C++ logging library*. Version 1.17.0. 2026. URL: https://github.com/gabime/spdlog.

[Mic26a] Microsoft Corporation. */EH (Exception Handling Model)*. 2026. URL: https://learn.microsoft.com/en-us/cpp/build/reference/eh-exception-handling-model.

[Mic26b] Microsoft Corporation. *Argument Passing and Naming Conventions*. 2026. URL: https://learn.microsoft.com/en-us/cpp/cpp/argument-passing-and-naming-conventions.

[Mic26c] Microsoft Corporation. *Detours: Software package*. Version 4.0.1. 2026. URL: https://github.com/microsoft/Detours.

[Mic26d] Microsoft Corporation. *Dynamic-Link Library Best Practices*. 2026. URL: https://learn.microsoft.com/en-us/windows/win32/dlls/dynamic-link-library-best-practices.

[Mic26e] Microsoft Corporation. *x64 ABI Conventions*. 2026. URL: https://learn.microsoft.com/en-us/cpp/build/x64-software-conventions.

[Mp26] Jupp Müller and prometheus-cpp Contributors. *prometheus-cpp: Prometheus Client Library for Modern C++*. Version 1.3.0. 2026. URL: https://github.com/jupp0r/prometheus-cpp.

[Pal24] Pallets Projects. *Jinja: A full-featured template engine for Python*. Version 3.1.6. 2024. URL: https://jinja.palletsprojects.com/.

[The26a] The MITRE Corporation. *Process Injection: Dynamic-link Library Injection, Sub-technique T1055.001*. 2026. URL: https://attack.mitre.org/techniques/T1055/001/.

[The26b] The Prometheus Authors. *Prometheus: An open-source systems monitoring and alerting toolkit*. 2026. URL: https://prometheus.io/.

[Yos+17] Pavel Yosifovich, Alex Ionescu, Mark E. Russinovich, and David A. Solomon. *Windows Internals, Part 1: System architecture, processes, threads, memory management, and more*. 7th. Microsoft Press, 2017. ISBN: 978-0-13-398634-1.